



INSTITUTO FEDERAL GOIANO
CAMPUS URUTAÍ
NÚCLEO DE INFORMÁTICA
ESPECIALIZAÇÃO EM INTELIGÊNCIA ARTIFICIAL APLICADA A DADOS CORPORATIVOS

LEONARDO MAXIMINO BERNARDO

**ACOLHEMENTE ESCOLAR PB: MOTOR DE
INFERÊNCIA LÓGICO PARA APOIO À GESTÃO
ESCOLAR NA PRIORIZAÇÃO RESPONSÁVEL
DE ACOLHIMENTO EM SAÚDE MENTAL**

Urutaí, 28 de maio de 2026

Sumário

1	APRESENTAÇÃO DO ALUNO E CONTEXTO	4
2	INTRODUÇÃO	5
3	DEFINIÇÃO DO PROBLEMA E OBJETIVOS	7
4	FUNDAMENTAÇÃO METODOLÓGICA	8
5	ARQUITETURA DE DADOS, PROVENIÊNCIA E GOVERNANÇA	10
6	ANÁLISE EXPLORATÓRIA E MOTIVAÇÃO EMPÍRICA .	16
7	REPRESENTAÇÃO DO CONHECIMENTO	18
8	BASE DE CONHECIMENTO, CNF E INFERÊNCIA	19
9	IMPLEMENTAÇÃO ACADÊMICA E MOTORES EQUIVALENTES	21
10	CENÁRIOS SINTÉTICOS E VALIDAÇÃO DO MOTOR . .	22
11	RESULTADOS, GUARDRAILS E COMPARAÇÃO COM BASELINE	25
12	EXPLICABILIDADE E GRAFOS DE RASTREABILIDADE	28
13	DISCUSSÃO CRÍTICA	30
14	PERSPECTIVA DE EVOLUÇÃO DO TRABALHO	32
15	CONCLUSÃO	34
	REFERÊNCIAS	35
	 APÊNDICES	 36
	APÊNDICE A – SÍMBOLOS PROPOSICIONAIS DO ACOLHEMENTE	37
	APÊNDICE B – REGRAS LÓGICAS E CNF	39

APÊNDICE C – MOTOR DE INFERÊNCIA POR RESOLUÇÃO	41
APÊNDICE D – CENÁRIOS SINTÉTICOS DE VALIDAÇÃO	44
APÊNDICE E – GRAFO DE EXPLICABILIDADE	47
APÊNDICE F – GERADOR DE TABELAS DE RESULTADOS	51
APÊNDICE G – EDA PENSE E DATASET EXTERNO . . .	54
APÊNDICE H – GRAFOS DE PROVENIÊNCIA E GOVERNANÇA	55
APÊNDICE I – FLUXO OPERACIONAL PSEUDONIMIZADO	69

1 Apresentação do Aluno e Contexto

Leonardo Maximino Bernardo é professor de Física da rede pública estadual da Paraíba desde 2020, com mais de 15 anos de experiência em docência nos níveis médio e superior. Doutor em Ciência e Engenharia de Materiais pela Universidade Federal da Paraíba, sua trajetória acadêmica combina modelagem computacional avançada e uma crescente atuação em ciência de dados e inteligência artificial.

Ao longo de sua formação, atuou como pesquisador na UFPB, desenvolvendo rotinas de cálculo e algoritmos para análise de grandes volumes de dados de simulação numérica de solidificação de ligas metálicas, extraindo padrões quantitativos para publicações acadêmicas. Essa experiência consolidou habilidades de pensamento analítico, resolução de problemas complexos e comunicação científica que informam diretamente sua prática docente e sua abordagem em projetos de inteligência artificial.

Atualmente, além da docência em Física, cursa Pós-Graduação *Lato Sensu* em Inteligência Artificial Aplicada pelo Instituto Federal Goiano e possui formação em Ciência de Dados pelo Unipê. Sua *stack* tecnológica inclui Python, JavaScript, SQL, *frameworks* como Django e React, e ferramentas de análise como Pandas, Scikit-learn e NetworkX. Possui certificações em desenvolvimento *back-end* com Node.js, Harvard CS50 e aceleração global de desenvolvimento de software.

O presente trabalho nasce da intersecção entre sua vivência como professor de escola pública estadual na Paraíba — onde testemunha diariamente demandas crescentes de acolhimento em saúde mental de adolescentes — e seu domínio técnico em modelagem computacional e lógica formal. A escolha do domínio de saúde mental escolar reflete o compromisso de aplicar inteligência artificial de forma ética, explicável e governada em um contexto de vulnerabilidade social, onde erros tecnológicos podem ter consequências humanas graves.

2 Introdução

A saúde mental de adolescentes no Brasil atravessa um momento crítico. Dados da Pesquisa Nacional de Saúde do Escolar — PeNSE 2024 (Instituto Brasileiro de Geografia e Estatística, 2024) — revelam que parcela significativa dos estudantes de 13 a 17 anos reporta sentimentos persistentes de tristeza, solidão e desesperança, com índices particularmente preocupantes na região Nordeste e, especificamente, no estado da Paraíba. O relatório da Organização Mundial da Saúde (World Health Organization, 2022) confirma essa tendência global: transtornos mentais respondem por uma proporção crescente da carga de doença entre jovens, e a maioria dos casos permanece sem qualquer forma de acolhimento adequado.

No contexto da escola pública estadual paraibana, a demanda por acolhimento em saúde mental frequentemente supera a capacidade de resposta da equipe pedagógica e psicossocial. Professores, coordenadores e orientadores educacionais enfrentam o desafio de identificar, dentre centenas de estudantes, aqueles que necessitam de atenção prioritária — sem dispor de ferramentas estruturadas para essa tarefa. O resultado é uma dependência excessiva de percepções individuais, sujeitas a viés cognitivo, fadiga decisória e inconsistência entre turnos e unidades escolares. Essa lacuna não é trivial: a ausência de priorização sistemática pode significar que um adolescente em situação de risco grave não receba acolhimento em tempo hábil.

Ao mesmo tempo, a adoção acrítica de tecnologias de inteligência artificial nesse domínio apresenta riscos igualmente sérios. Sistemas baseados em aprendizado de máquina ou IA generativa, embora poderosos em outros contextos, introduzem opacidade decisória incompatível com o rigor ético exigido quando se trata de menores de idade em situação de vulnerabilidade (JOBIN; IENCA; VAYENA, 2019). A Lei Geral de Proteção de Dados — LGPD (Brasil, 2018) — e o Estatuto da Criança e do Adolescente — ECA (Brasil, 1990) — impõem salvaguardas rigorosas para o tratamento de dados sensíveis de menores, incluindo o direito à explicação de decisões automatizadas. Um modelo de “caixa preta” que classifique ou ranqueie estudantes por risco psicológico seria, no mínimo, juridicamente temerário e, no limite, eticamente inaceitável.

Diante desse cenário, o presente trabalho propõe o **AcolheMente Escolar PB**: um Sistema Baseado em Conhecimento que utiliza Lógica Proposicional com Inferência por Resolução para apoiar — e nunca substituir — a gestão escolar na priorização responsável de acolhimento em saúde mental. A escolha por lógica simbólica determinística não é uma limitação técnica, mas uma decisão de *design* ético: garante explicabilidade total, rastreabilidade de cada conclusão e ausência de qualquer linguagem diagnóstica clínica (RUSSELL; NORVIG, 2022).

Este documento está organizado da seguinte forma: o Capítulo 3 define formalmente

o problema e os objetivos; o Capítulo 4 apresenta a fundamentação metodológica; o Capítulo 5 detalha a arquitetura de dados e proveniência; o Capítulo 6 apresenta a análise exploratória e a motivação empírica; o Capítulo 7 trata da representação do conhecimento; o Capítulo 8 unifica base de conhecimento, CNF e inferência; o Capítulo 9 descreve a implementação acadêmica; o Capítulo 10 explica os cenários sintéticos; o Capítulo 11 apresenta resultados e *guardrails*; o Capítulo 12 discute explicabilidade; o Capítulo 13 oferece discussão crítica; o Capítulo 14 analisa perspectivas de evolução; e o Capítulo 15 conclui o trabalho.

3 Definição do Problema e Objetivos

O problema central que este trabalho endereça pode ser formulado da seguinte maneira: como apoiar a gestão escolar de escolas públicas estaduais da Paraíba na priorização sistemática de acolhimento em saúde mental de adolescentes, utilizando inteligência artificial explicável, sem produzir diagnósticos clínicos, sem violar a LGPD ou o ECA, e sem substituir o julgamento humano profissional?

O escopo da solução proposta compreende a construção de um motor de inferência lógico determinístico com 7 variáveis proposicionais — 6 variáveis de entrada e 1 variável de saída — e 6 regras em Forma Normal Conjuntiva, ancorado conceitualmente em indicadores agregados da PeNSE 2024 e validado com cenários sintéticos determinísticos. O sistema não diagnostica, não classifica alunos individualmente, não utiliza dados identificáveis, não substitui avaliação profissional de psicólogos ou assistentes sociais, e não se conecta a APIs de IA generativa.

O **objetivo geral** é desenvolver e validar um motor de inferência por resolução, baseado em lógica proposicional e fundamentado em dados oficiais da PeNSE 2024, para apoiar a priorização de acolhimento em saúde mental escolar no estado da Paraíba.

Os **objetivos específicos** são:

1. Modelar o domínio de saúde mental escolar em 7 variáveis proposicionais (4 entradas nucleares, 2 entradas contextuais e 1 saída inferida), ancoradas em variáveis da PeNSE 2024.
2. Formalizar 6 regras de inferência em Forma Normal Conjuntiva e implementar motor de resolução determinístico.
3. Validar o motor com cenários sintéticos representativos, cobrindo todas as $2^6 = 64$ combinações possíveis das 6 variáveis de entrada, e comparar com *baseline* manual.
4. Garantir conformidade integral com LGPD, ECA e Programa Saúde na Escola (Brasil, 2007).
5. Documentar explicabilidade, limitações e perspectivas de evolução com fundamentação metodológica.

4 Fundamentação Metodológica

O AcolheMente Escolar PB é, em sua essência, um Sistema Baseado em Conhecimento (SBC) — uma classe de sistemas de inteligência artificial que codifica explicitamente o conhecimento de especialistas humanos em regras formais e utiliza mecanismos de inferência para derivar conclusões a partir de fatos observados (BRACHMAN; LEVESQUE, 2004). Diferentemente de abordagens estatísticas ou conexionistas, um SBC não “aprende” a partir de dados brutos: opera sobre uma base de conhecimento construída deliberadamente por engenheiros do conhecimento em colaboração com especialistas do domínio.

Essa característica é particularmente valiosa no contexto de saúde mental escolar, onde o volume de dados individuais disponível é insuficiente para treinamento de modelos de *machine learning*, a transparência de cada decisão é requisito ético e legal, e o custo de um erro — falso positivo que rotula indevidamente um estudante, ou falso negativo que ignora um caso grave — é inaceitavelmente alto.

A técnica específica adotada é a Lógica Proposicional com Inferência por Resolução. Trata-se de um formalismo da lógica clássica no qual o conhecimento é representado por proposições atômicas e conectivos lógicos. A inferência é realizada por meio do método de resolução, um procedimento correto e refutacionalmente completo para a lógica proposicional (GENESERETH; NILSSON, 1987). O método opera sobre cláusulas em Forma Normal Conjuntiva: para verificar se uma conclusão A pode ser derivada de uma base de conhecimento KB , o procedimento adiciona a negação da conclusão ($\neg A$) ao conjunto de cláusulas e aplica repetidamente a regra de resolução, buscando derivar a cláusula vazia, o que constitui prova por contradição de que $KB \models A$.

A Trilha I do curso de Pós-Graduação em IA Aplicada do IF Goiano tem como foco técnicas simbólicas e de busca. A escolha por lógica proposicional com resolução é duplamente justificada: atende ao escopo pedagógico da trilha e oferece as propriedades de explicabilidade e determinismo exigidas pelo domínio.

Foram consideradas e descartadas alternativas como Lógica de Primeira Ordem, que introduziria complexidade desnecessária para um domínio com 7 variáveis binárias; Redes Bayesianas, que exigiriam distribuições de probabilidade condicional inestimáveis a partir de dados agregados; Árvores de Decisão, que dependem de treinamento supervisionado com dados rotulados inexistentes; Redes Neurais, incompatíveis com os requisitos de explicabilidade total; e IA Generativa, descartada por não oferecer garantias de consistência lógica e determinismo.

Conforme argumentam Russell e Norvig (2022), a escolha da representação do conhecimento deve ser guiada pelas propriedades do domínio, e não pela sofisticação da técnica. No caso do AcolheMente Escolar PB, a lógica proposicional é a ferramenta *correta*, não uma ferramenta *limitada*.

O princípio de *human-in-the-loop* é inegociável. A variável de saída A é um sinal para o profissional humano, não uma decisão final. O fluxo operacional previsto é: o motor sinaliza priorização; a equipe pedagógica avalia o caso; a decisão de ação é sempre humana.

5 Arquitetura de Dados, Proveniência e Governança

A credibilidade de qualquer sistema de inteligência artificial aplicado a populações vulneráveis depende, antes de tudo, da rastreabilidade e da legitimidade dos dados que o sustentam. No AcolheMente Escolar PB, essa preocupação se traduz em uma arquitetura de proveniência de dados dividida em três camadas mutuamente exclusivas, cada uma com finalidade, escopo e restrições de uso rigorosamente definidos. A separação não é apenas uma decisão técnica: é um requisito de governança que impede contaminação cruzada entre fontes de naturezas distintas.

A primeira camada, denominada TIER_A, corresponde à PeNSE 2024, base de dados oficial e de acesso público do IBGE (Instituto Brasileiro de Geografia e Estatística, 2024). A PeNSE é uma pesquisa amostral com representatividade nacional, regional e por unidade da federação. Os microdados utilizados referem-se exclusivamente ao Tema 12 (Saúde Mental) e ao Tema 01 (Informações Gerais), com recorte para o estado da Paraíba. O uso autorizado desta camada é estritamente analítico: a PeNSE sustenta a Análise Exploratória de Dados, contextualiza o problema epidemiológico e orienta a escolha das variáveis proposicionais. Os dados da PeNSE **não alimentam diretamente o motor de inferência** para fins de classificação individual, pois a PeNSE é pesquisa populacional, não sistema de acompanhamento escolar ativo, e seus respondentes não são sujeitos de intervenção direta neste projeto.

A segunda camada, TIER_B, compreende cenários sintéticos gerados computacionalmente para validação do motor lógico. Cada cenário é um vetor binário de 6 posições, uma para cada variável de entrada (E, B, V, S, C, I), representando combinações hipotéticas de indicadores. Nenhum dado real de aluno ou escola é utilizado nesta camada. Em um sistema baseado em conhecimento com lógica proposicional, cenários sintéticos controlados testam as fronteiras das regras com rigor superior ao que dados ruidosos permitiriam, pois cada cenário corresponde a uma valoração booleana precisa e determinística.

A terceira camada, TIER_C, é um *dataset* externo utilizado exclusivamente para *benchmark* metodológico — para validar que o *pipeline* de processamento funciona corretamente com dados tabulares de outra origem. O *dataset* externo não gera conclusões sobre o Brasil, o Nordeste ou a Paraíba. Qualquer tentativa de inferência regional a partir do TIER_C é logicamente inválida e explicitamente proibida.

O *merge* entre tiers é estritamente proibido. Essa barreira garante que conclusões derivadas de dados oficiais nunca sejam contaminadas por dados sintéticos ou externos, e vice-versa. A violação dessa regra invalidaria simultaneamente a integridade científica e a conformidade jurídica do sistema.

A LGPD (Brasil, 2018) classifica dados de saúde de menores como dados pessoais sensíveis, impondo requisitos particularmente rigorosos. O AcolheMente atende a esses requisitos *by design*: não processa dados pessoais identificáveis, opera sobre indicadores agregados anonimizados, toda conclusão é rastreável a regras explícitas, e não produz decisões automatizadas sobre indivíduos. O ECA (Brasil, 1990) veda qualquer forma de rotulagem, classificação ou estigmatização de crianças e adolescentes. O Programa Saúde na Escola (Brasil, 2007) estabelece a integração entre escola e rede de saúde como política pública; o AcolheMente apoia essa integração, oferecendo à equipe escolar um instrumento de priorização que indica quando acionar os fluxos de encaminhamento previstos no PSE.

A justificativa para não utilizar os dados reais da PeNSE como entradas individuais do motor merece elaboração adicional, pois constitui uma das decisões arquiteturais mais importantes do projeto. Utilizar respostas individuais de saúde mental para inferência operacional sem possibilidade de acolhimento real violaria o princípio de finalidade da LGPD e poderia produzir uma forma invisível de classificação sensível — um “diagnóstico silencioso” que nenhum profissional de saúde solicitou e nenhum adolescente consentiu. Por essas razões, o motor é validado por cenários sintéticos determinísticos, enquanto a PeNSE sustenta a justificativa epidemiológica e a modelagem conceitual.

Tabela 1 – Camadas de proveniência e uso autorizado.

Tier	Fonte	Uso permitido	Uso proibido	Justificativa
A	PeNSE 2024	EDA, motivação	Inferência individual	Pesquisa populacional
B	Sintéticos	Validação do motor	Representar alunos	Controle determinístico
C	Externo	Benchmark	Conclusão sobre PB	Sem validade regional

A Figura 1 visualiza o fluxo completo de proveniência, evidenciando que cada tier alimenta exclusivamente seu destino autorizado, com barreira explícita de *merge*.

O diagrama ilustra o processo de validação de cenários de projeto, organizado em três níveis (Tiers) e seus respectivos resultados.

TIER A (Azul):

- PeNSE 2024 (IBGE)**: Dados oficiais anonimizados.
- EDA / Motivação**: Prevalência PB vs NE vs BR. Justifica o problema.
- RESULTADO**: Variáveis proporcionais. E, B, V, S, C, I, A.

TIER B (Laranja):

- Cenários sintéticos**: Criados pelo projeto.
- Motor lógico**: Inferência por resolução.
- Validação R1-R6**: 64 cenários testados. 0 falhas.

TIER C (Púrpura):

- Dataset externo**: Benchmark metodológico.
- EDA exploratória**: Extensibilidade metodológica.
- NAO inferência**: Sem validade para Brasil/Paraná (destacado com uma caixa tracejada vermelha).

Uma linha tracejada vermelha separa o TIER B do TIER C, com o texto **MERGE ENTRE TIERS PROIBIDO** no centro.

Anonimização, pseudonimização e reidentificação restrita

A anonimização irreversível, prevista no art. 12 da LGPD (Brasil, 2018), é adequada para relatórios, painéis agregados e pesquisa, pois elimina qualquer possibilidade de vincular o dado ao indivíduo. Entretanto, se os dados forem anonimizados de forma irreversível, a escola não conseguirá saber qual estudante acolher. A anonimização resolve o problema da privacidade, mas inviabiliza a finalidade pedagógica e psicossocial do sistema: não há acolhimento possível sem saber quem acolher.

A reidentificação não seria automática: exigiria solicitação formal por membro da

equipe autorizada, com registro auditável de quem acessou, quando e por quê. Professores poderiam registrar observações pedagógicas objetivas em formulário estruturado, se autorizados, mas não devem ter acesso amplo a *scores* sensíveis, diagnósticos ou chaves de reidentificação. O painel analítico visível para a gestão escolar exibiria apenas dados agregados por turma, série ou turno, com supressão de grupos pequenos para impedir identificação indireta. O motor, o relatório e o painel geral nunca devem exibir nome, matrícula, CPF, telefone, endereço ou qualquer forma de *ranking* individual.

A Tabela 2 detalha a matriz de papéis e permissões proposta para um piloto operacional pseudonimizado, distinguindo cinco perfis de acesso com responsabilidades e restrições explícitas.

Tabela 2 – Perfis de acesso em piloto operacional pseudonimizado.

Perfil	Pode visualizar	Não pode visualizar	Finalidade
Professor	Orientações pedagógicas gerais; formulário de observação objetiva	<i>Ranking</i> ; <i>score</i> sensível; diagnóstico; dados clínicos; chave de reidentificação	Observação pedagógica e cuidado cotidiano
Coordenação / orientação	Caso pseudonimizado; regras acionadas; justificativa de priorização	Dados além do mínimo necessário	Triagem humana e acolhimento inicial
Direção / equipe designada	Chave de reidentificação sob custódia; registro auditável de acesso; encaminhamento	Uso livre ou indiscriminado dos dados	Responsabilidade institucional e acionamento de fluxos
PSE / UBS / CAPS	Encaminhamento humano qualificado; informações necessárias ao cuidado	Resultado bruto automatizado sem mediação	Cuidado em rede
Painel analítico	Dados agregados por turma/série/turno; indicadores com supressão de grupos pequenos	Nome; matrícula; CPF; telefone; <i>ranking</i> individual	Gestão preventiva e planejamento

Essa arquitetura de papéis reflete o princípio de minimização de acesso: cada ator vê apenas o que é estritamente necessário para exercer sua função (Brasil, 2018). A expressão “a gestão pode acessar” não é suficiente: o correto é definir *qual* membro da equipe, em *qual* papel, pode acessar *qual* dado, com *qual* registro de auditoria. O ECA (Brasil, 1990) reforça que a proteção integral de crianças e adolescentes exige cautela

redobrada no acesso a informações sensíveis. O Programa Saúde na Escola (Brasil, 2007) prevê articulação entre escola e rede de saúde — o AcolheMente pode alimentar esse fluxo, desde que a identificação do estudante seja mediada por equipe autorizada e não exposta em qualquer painel ou relatório automatizado.

A Figura 2 visualiza o fluxo operacional completo, desde a observação dos indicadores mínimos até o acolhimento e o registro auditável, com destaque para os nós de proibição que impedem exposição de dados.



Figura 2 – Fluxo operacional pseudonimizado: o motor processa apenas `case_id` e variáveis booleanas. A chave de reidentificação permanece sob custódia da equipe designada.

É importante enfatizar que essa arquitetura de pseudonimização não altera o motor lógico, as regras R1–R6, a CNF ou os cenários sintéticos. Trata-se de uma camada de governança de identidade que envolve o motor, sem modificá-lo. A versão acadêmica apresentada neste documento não implementa pseudonimização porque não processa estudantes reais. A discussão é conceitual e arquitetural, preparando o terreno para uma eventual transição operacional responsável.

6 Análise Exploratória e Motivação Empírica

A Análise Exploratória de Dados cumpre papel fundamental neste trabalho: é ela que transforma a percepção qualitativa do autor — “há sofrimento emocional crescente entre os alunos” — em evidência quantitativa sustentada por dados oficiais. A EDA não alimenta o motor de inferência; ela justifica sua existência.

Os microdados da PeNSE 2024 (Instituto Brasileiro de Geografia e Estatística, 2024) foram explorados com foco no Tema 12 (Saúde Mental) e no Tema 01 (Informações Gerais), com recorte para o estado da Paraíba. As variáveis centrais analisadas cobrem quatro dimensões do sofrimento adolescente: as variáveis B12004, B12005 e B12007, que mensuram sentimentos de tristeza, solidão e desesperança, foram operacionalizadas na variável proposicional *E* (sofrimento emocional recorrente); as variáveis B12003 e B07004, que capturam percepção de apoio socioafetivo e amizades próximas, foram operacionalizadas na variável *B* (baixo apoio socioafetivo percebido); a variável B12008, que registra sentimentos persistentes de desvalor da vida, corresponde à variável *V* (indicador crítico de desvalor da vida); e a variável B12009, que registra relato de autoagressão, corresponde à variável *S* (sinal autorreferido de autoagressão). Além das variáveis nucleares, os indicadores B03010C e B03006B foram operacionalizados na variável contextual *C* (contexto comportamental agravante), e os indicadores E01P60 e E01P117 na variável contextual *I* (insuficiência institucional de resposta).

A análise exploratória da PeNSE 2024 é utilizada neste trabalho como referência epidemiológica e motivação para a escolha das variáveis proposicionais. Os indicadores de saúde mental do Tema 12, analisados com recorte para a Paraíba, sugerem prevalências relevantes de sofrimento emocional adolescente na região. A escolha de cada variável proposicional decorreu dessa análise, identificando quais dimensões apresentam maior relevância para o domínio modelado. Na versão acadêmica aqui apresentada, esses dados não são usados para classificação individual nem substituem validação operacional em escola real.

A EDA foi implementada no módulo `src/acolhemente/eda_plots.py`, que contém funções dedicadas para cada tier de dados, com separação estrita entre TIER_A e TIER_C. A função `plot_pense_pb_vs_ne_br()` gera gráficos comparativos entre Paraíba, Nordeste e Brasil para os indicadores centrais de saúde mental. A função `plot_external_behavioral_distribution()` trata exclusivamente do *dataset* externo (TIER_C), com aviso obrigatório de que os dados não possuem validade inferencial para o contexto brasileiro.

É fundamental compreender que a EDA orienta a escolha das proposições, mas não decide casos. A análise exploratória revelou quais indicadores da PeNSE são mais relevantes para a Paraíba, e essa relevância foi traduzida na seleção das 7 variáveis proposicionais. Contudo, a EDA não gera diagnóstico, não classifica indivíduos e não alimenta o motor

— ela fundamenta o problema, não resolve casos.

O *dataset* externo (TIER_C) foi utilizado exclusivamente para *benchmark* metodológico, verificando que o *pipeline* de processamento funciona corretamente com dados tabulares de outra origem. Toda visualização gerada a partir do TIER_C carrega aviso explícito de ausência de validade inferencial para o Brasil ou a Paraíba. As funções de EDA do TIER_C estão isoladas no módulo `eda_plots.py` e nunca acessam dados do TIER_A, garantindo a integridade da barreira de proveniência.

7 Representação do Conhecimento

A modelagem do domínio seguiu princípios clássicos de engenharia do conhecimento (BRACHMAN; LEVESQUE, 2004): identificação de conceitos relevantes, elicitacão de regras com especialistas, formalizacão em lógica proposicional e validacão iterativa. O resultado é um conjunto de 7 variáveis proposicionais, organizado em 4 entradas nucleares, 2 entradas contextuais e 1 saída inferida, que representam o espaço de estados relevante para a priorizacão de acolhimento.

O modelo possui 7 variáveis proposicionais no total, sendo **6 variáveis de entrada** (E, B, V, S, C, I) e **1 variável de saída** (A). Essa distinção é fundamental: A não é uma variável observada, mas o resultado da inferência lógica do motor. Portanto, o espaço combinatório de estados de entrada é $2^6 = 64$, não $2^7 = 128$.

A decisão de utilizar exatamente 7 variáveis foi deliberada. Com 5 variáveis, perder-se-ia a capacidade de modelar fatores contextuais que modulam a urgência do acolhimento. Com mais de 7, o espaço combinatório cresceria sem ganho proporcional de discriminação, violando o princípio de parcimônia que governa sistemas simbólicos interpretáveis.

As quatro entradas nucleares são: E (sofrimento emocional recorrente, fontes PeNSE B12004, B12005, B12007), B (baixo apoio socioafetivo percebido, fontes B12003, B07004), V (indicador crítico de desvalor da vida, fonte B12008) e S (sinal autorreferido de autoagressão, fonte B12009). As duas entradas contextuais são: C (contexto comportamental agravante, fontes B03010C, B03006B) e I (insuficiência institucional de resposta, fontes E01P60, E01P117). A variável A é a saída inferida do motor, representando acolhimento humano prioritário. As variáveis contextuais nunca inferem A isoladamente. C só participa de regras em conjunção com E e B ; I só participa em conjunção com V . Essa restrição é um *guardrail* lógico que impede falsos positivos por fatores contextuais.

O sistema não diagnostica condições clínicas. As variáveis representam indicadores comportamentais observáveis e auto-reportados, não categorias nosológicas. A variável A sinaliza necessidade de acolhimento pedagógico e psicossocial — uma ação escolar, não uma intervenção clínica.

Tabela 3 – Variáveis proposicionais do AcolheMente Escolar PB.

Var	Tipo	Semântica Operacional	Fontes PeNSE
E	Entrada nuclear	Sofrimento emocional recorrente	B12004, B12005, B12007
B	Entrada nuclear	Baixo apoio socioafetivo percebido	B12003, B07004
V	Entrada nuclear	Indicador de desvalor da vida	B12008
S	Entrada nuclear crítica	Sinal de autoagressão	B12009
A	Saída inferida	Acolhimento prioritário	Inferida
C	Entrada contextual	Contexto comportamental agravante	B03010C, B03006B
I	Entrada contextual	Insuficiência institucional	E01P60, E01P117

8 Base de Conhecimento, CNF e Inferência

A base de conhecimento do AcolheMente é composta por 6 regras de inferência, cada uma representando um cenário em que a combinação de indicadores justifica a priorização de acolhimento. As regras foram definidas com base em princípios pedagógicos e psicossociais, não em correlações estatísticas.

A regra R1, $S \rightarrow A$, estabelece que a presença de sinal autorreferido de autoagressão é suficiente para acionar incondicionalmente o fluxo de acolhimento. Esta é a regra de maior criticidade: autoagressão é o indicador mais grave do espectro modelado, e qualquer atraso no acolhimento pode ter consequências irreversíveis. A regra R2, $V \wedge E \rightarrow A$, captura a coocorrência de desvalor da vida e sofrimento emocional recorrente, indicando um quadro de vulnerabilidade emocional persistente. As regras R3 ($E \wedge B \rightarrow A$) e R4 ($V \wedge B \rightarrow A$) modelam cenários em que sofrimento emocional ou desvalor da vida se combinam com baixo apoio socioafetivo, configurando situações em que o estudante não dispõe de rede de suporte.

A regra R5, $E \wedge B \wedge C \rightarrow A$, registra o contexto comportamental agravante como marcador explicativo subsumido por R3, enquanto R6, $V \wedge I \rightarrow A$, combina desvalor da vida com insuficiência institucional de resposta. Note-se que as variáveis contextuais (C e I) só participam em conjunção com variáveis nucleares, nunca isoladamente. A variável A é a única conclusão derivável pelo motor, refletindo o princípio de que o sistema deve emitir um sinal binário claro, sem gradações ou categorizações intermediárias que possam ser mal interpretadas pela equipe escolar.

Tabela 4 – Regras lógicas R1–R6 e respectivas cláusulas CNF.

Regra	Forma Implicativa	CNF
R1	$S \rightarrow A$	$\neg S \vee A$
R2	$V \wedge E \rightarrow A$	$\neg V \vee \neg E \vee A$
R3	$E \wedge B \rightarrow A$	$\neg E \vee \neg B \vee A$
R4	$V \wedge B \rightarrow A$	$\neg V \vee \neg B \vee A$
R5	$E \wedge B \wedge C \rightarrow A$	$\neg E \vee \neg B \vee \neg C \vee A$
R6	$V \wedge I \rightarrow A$	$\neg V \vee \neg I \vee A$

Para aplicar o algoritmo de inferência por resolução, todas as regras implicativas devem ser convertidas para Forma Normal Conjuntiva — uma conjunção de cláusulas disjuntivas de literais. A transformação segue a equivalência lógica fundamental $p \rightarrow q \equiv \neg p \vee q$. Para regras com antecedentes conjuntivos, a equivalência generaliza-se: $p \wedge q \rightarrow r \equiv \neg p \vee \neg q \vee r$. Esse procedimento é correto e preserva a validade lógica das regras originais (RUSSELL; NORVIG, 2022).

O método de resolução é um procedimento de prova por refutação: para verificar se $KB \models A$, adiciona-se $\neg A$ ao conjunto de cláusulas e aplica-se iterativamente a regra de resolução. Se a cláusula vazia $\square$ for derivada, conclui-se que $KB \models A$; caso contrário, A não pode ser derivada. A regra de resolução opera sobre pares de cláusulas que contenham literais complementares. O procedimento é completo para a lógica proposicional: se $KB \models A$, o método sempre encontra a prova em tempo finito.

Listing 8.1 – Pseudocódigo do motor de resolução.

```
1 funcao RESOLUCAO(KB, conclusao):
2   clausulas <- KB uniao {NOT conclusao}
3   repita:
4     novas <- conjunto vazio
5     para cada par (Ci, Cj) em clausulas:
6       resolvente <- RESOLVER(Ci, Cj)
7       se resolvente = clausula_vazia:
8         retorna VERDADEIRO
9     novas <- novas uniao {resolvente}
10    se novas subconjunto de clausulas:
11      retorna FALSO
12    clausulas <- clausulas uniao novas
```

A complexidade computacional do método de resolução para lógica proposicional é exponencial no pior caso, mas para a base de conhecimento do AcolheMente (6 cláusulas com no máximo 4 literais cada), a convergência é virtualmente instantânea.

9 Implementação Acadêmica e Motores Equivalentes

O motor de inferência foi implementado em Python, seguindo princípios de modularidade e testabilidade. A arquitetura separa três responsabilidades: representação das cláusulas CNF (Apêndice A–B), mecanismo de resolução (Apêndice C) e interface de validação com cenários sintéticos (Apêndice D). A implementação adota o padrão de representação de cláusulas como conjuntos imutáveis (`frozenset`) de literais, onde literais positivos são representados pelo nome da variável e literais negativos pelo prefixo `~`. Essa escolha garante que cláusulas sejam *hashable* e possam ser armazenadas em conjuntos, evitando duplicatas durante o processo de resolução.

O projeto mantém duas versões do motor de inferência, ambas matematicamente equivalentes. A versão de produção, localizada em `src/acolhemente/motor.py`, possui arquitetura modular com camadas de abstração, integração com o grafo de explicabilidade e *pipeline* de testes automatizados; é a versão que seria utilizada em eventual implantação institucional. A versão acadêmica, localizada em `appendices/motor_resolucao_acolhemente.py`, é uma simplificação didática com menos de 100 linhas de código, sem dependências externas além da biblioteca padrão, com comentários explicativos em português. É a versão apresentada nos Apêndices deste PDF e executada no Google Colab.

A simplificação do motor acadêmico não significa regra diferente. As regras R1–R6, a CNF e a variável de saída A são idênticas em ambas as versões. A diferença reside em aspectos de engenharia de software: menos camadas de arquitetura, menos abstrações orientadas a objetos, mais comentários didáticos para leitura direta no PDF, e execução simples no Colab sem necessidade de instalação de pacotes. A escolha de apresentar a versão acadêmica no PDF segue o padrão do modelo de resposta da Trilha I, que privilegia scripts didáticos em apêndices executáveis. Um teste de equivalência (`tests/test_motor_equivalence_contract.py`) verifica que ambas as versões produzem saídas idênticas para os mesmos cenários de entrada.

O motor inclui função de explicabilidade que, para cada conclusão $A = \text{verdadeiro}$, identifica quais regras foram acionadas e produz texto explicativo em linguagem natural. Essa funcionalidade é essencial para o princípio de *human-in-the-loop*: a equipe escolar pode verificar não apenas *se* o acolhimento foi priorizado, mas *por que*. O código-fonte completo encontra-se nos Apêndices A–F.

10 Cenários Sintéticos e Validação do Motor

O modelo possui 7 variáveis proposicionais no total, sendo 6 variáveis de entrada (E , B , V , S , C , I) e 1 variável de saída (A). Portanto, existem $2^6 = 64$ combinações possíveis das variáveis de entrada. Os cenários sintéticos não representam alunos reais: existem exclusivamente para testar a fronteira lógica de cada regra em condições controladas.

Em um Sistema Baseado em Conhecimento com lógica proposicional, cenários sintéticos determinísticos são a ferramenta adequada de validação. Diferentemente de modelos estatísticos — que exigem grandes volumes de dados para estimação de parâmetros —, sistemas simbólicos podem ser validados exaustivamente por enumeração do espaço de estados. Cada cenário corresponde a uma valoração booleana controlada das 6 variáveis de entrada, permitindo verificar se o motor produz a saída esperada para cada combinação.

A razão pela qual não se utiliza a PeNSE diretamente no motor já foi discutida no Capítulo 5, mas merece reiteração neste contexto: a PeNSE é pesquisa populacional, não prontuário individual. Seus dados agregados justificam a existência do problema e orientam a escolha das variáveis, mas não constituem entradas operacionais para inferência lógica sobre indivíduos. Os cenários sintéticos, por outro lado, foram projetados deliberadamente para testar cada fronteira lógica de cada regra, oferecendo cobertura que dados reais ruidosos não proporcionariam com a mesma precisão.

Os cenários foram criados seguindo uma lógica sistemática. O cenário nulo, com todas as variáveis falsas, verifica que o motor não emite alertas espontâneos — ausência de falsos positivos em condições de quietude. O cenário de S isolado testa a regra R1, a mais crítica do sistema, confirmando que autoagressão dispara acolhimento incondicional. Os cenários de combinações binárias ($V + E$, $E + B$, $V + B$) testam as regras R2, R3 e R4, verificando que pares de variáveis nucleares acionam corretamente o motor. O cenário $E + B + C$ evidencia que R5 é acionada conjuntamente com R3, enriquecendo a explicação sem alterar o resultado decisório. O cenário $V + I$ testa R6, verificando a interação entre desvalor da vida e insuficiência institucional.

Os *guardrails* foram testados com particular rigor. O cenário de C isolado, com todas as demais variáveis falsas, verifica que contexto agravante sozinho não estigmatiza um estudante — o motor retorna “não”, confirmando que a restrição lógica funciona. O cenário de I isolado aplica o mesmo teste para insuficiência institucional. O cenário $C + I$ sem variáveis nucleares verifica que mesmo a combinação de ambos os fatores contextuais não é suficiente para acionar o acolhimento. A variável S é a única exceção deliberada ao princípio de que nenhuma variável isolada aciona A : autoagressão é, por definição, um indicador de gravidade incondicional.

Os cenários de variáveis nucleares isoladas (E sozinho, B sozinho, V sozinho) confirmam que nenhuma variável nuclear, exceto S , é suficiente isoladamente para acionar

o motor. Essa característica é deliberada: o sistema exige combinações específicas para evitar falsos positivos por sinais ambíguos. O cenário com todas as variáveis verdadeiras testa a saturação total, confirmando que múltiplas regras podem ser acionadas simultaneamente sem contradição lógica.

A cobertura exaustiva das $2^6 = 64$ combinações garante que não há “pontos cegos” no motor. Cada combinação possível foi testada e cada resultado é determinístico e rastreável à regra que o originou. A tabela a seguir apresenta os cenários representativos selecionados, com a finalidade de cada teste explicitada.

Tabela 5 – Cenários sintéticos: valoração, resultado e finalidade.

Cenário	<i>E</i>	<i>B</i>	<i>V</i>	<i>S</i>	<i>C</i>	<i>I</i>	<i>A</i>	Regra	Finalidade
Nulo	0	0	0	0	0	0	NÃO	—	Ausência de falso positivo
S isolado	0	0	0	1	0	0	SIM	R1	Autoagressão incondicional
V+E	1	0	1	0	0	0	SIM	R2	Desvalor + sofrimento
E+B	1	1	0	0	0	0	SIM	R3	Sufrimento + baixo apoio
V+B	0	1	1	0	0	0	SIM	R4	Desvalor + baixo apoio
E+B+C	1	1	0	0	1	0	SIM	R5	Contexto agravante
V+I	0	0	1	0	0	1	SIM	R6	Insuf. institucional
C isolado	0	0	0	0	1	0	NÃO	—	<i>Guardrail C</i>
I isolado	0	0	0	0	0	1	NÃO	—	<i>Guardrail I</i>
C+I	0	0	0	0	1	1	NÃO	—	<i>Guardrail</i> duplo
Todas	1	1	1	1	1	1	SIM	R1–R6	Saturação total
E isolado	1	0	0	0	0	0	NÃO	—	Nuclear insuficiente
B isolado	0	1	0	0	0	0	NÃO	—	Nuclear insuficiente
V isolado	0	0	1	0	0	0	NÃO	—	Nuclear insuficiente

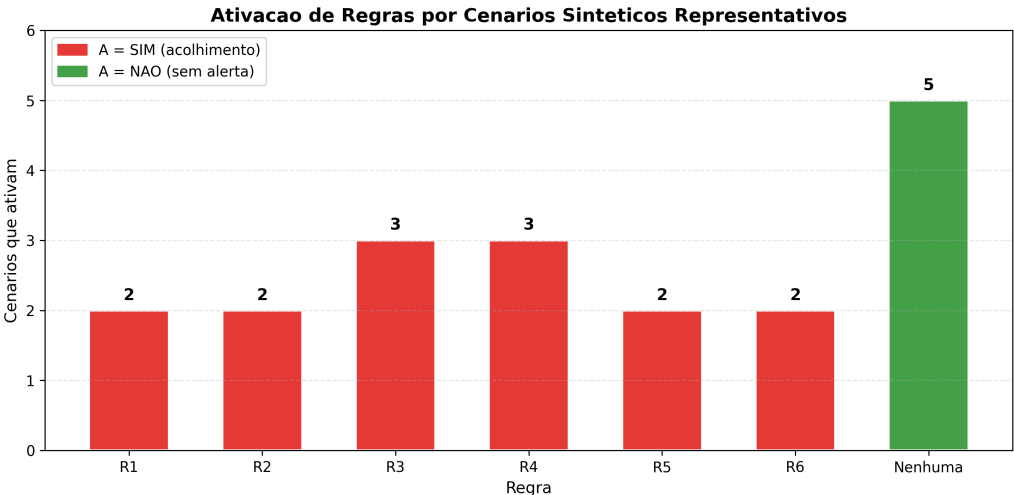


Figura 3 – Ativação de regras por cenários sintéticos representativos.

Cabe registrar uma propriedade lógica relevante da base de conhecimento: a regra R5 ($E \wedge B \wedge C \rightarrow A$) é **subsumida** pela regra R3 ($E \wedge B \rightarrow A$). Isso significa que, em toda valoração em que R5 dispara — isto é, quando E , B e C são todos verdadeiros —, R3 também dispara, pois E e B já são suficientes. A validação exaustiva das $2^6 = 64$ combinações confirma que não há nenhum cenário em que R5 seja a *única* regra acionada. Apesar da subsunção, R5 é deliberadamente mantida na base por três razões: (1) enriquece a explicação ao profissional humano, incluindo o fator contextual C na lista de regras acionadas; (2) prepara a base para futuras versões com lógica *fuzzy*, em que C poderia modular graus de urgência; e (3) não introduz falsos positivos nem altera o resultado de nenhum cenário. O registro completo das 64 combinações encontra-se no arquivo `outputs/validacao_exaustiva_64.csv`.

11 Resultados, Guardrails e Comparação com Baseline

Os resultados da validação exaustiva com as $2^6 = 64$ combinações possíveis das variáveis de entrada confirmam o comportamento esperado em todos os cenários testados, sem exceção. Nenhum falso positivo foi observado no cenário nulo — quando todas as variáveis são falsas, o motor corretamente não infere A , demonstrando que o sistema não emite alertas espontâneos. A variável S (autoagressão) é suficiente para acionar o acolhimento isoladamente, confirmando a prioridade incondicional da regra R1. As combinações binárias $V + E$, $E + B$ e $V + B$ ativam corretamente R2, R3 e R4, enquanto as combinações com contextuais $E + B + C$ e $V + I$ ativam R5 e R6 conforme projetado.

Os testes de *guardrail* merecem destaque especial, pois constituem a demonstração mais importante da responsabilidade ética do sistema. Quando apenas C é verdadeira, com todas as demais falsas, o motor retorna “não” — um estudante inserido em contexto comportamental negativo não é automaticamente rotulado como prioritário, o que evitaria estigmatização injusta. O mesmo resultado obtém-se quando apenas I é verdadeira: insuficiência institucional não é, por si só, motivo para sinalizar um adolescente. O teste mais rigoroso é o cenário $C + I$ sem variáveis nucleares: mesmo com ambos os fatores contextuais ativos, o motor não infere A , confirmando que a combinação de contextuais não supera o *guardrail* lógico. Esses resultados demonstram que o sistema foi projetado para proteger, não para classificar.

Os cenários de variáveis nucleares isoladas complementam a validação: E sozinho, B sozinho e V sozinho não acionam A . Essa característica confirma que o motor exige combinações específicas de indicadores para evitar falsos positivos por sinais isolados e potencialmente ambíguos. O cenário com todas as variáveis verdadeiras demonstra que múltiplas regras podem ser acionadas simultaneamente (R1 a R6) sem contradição lógica, confirmando a consistência da base de conhecimento.

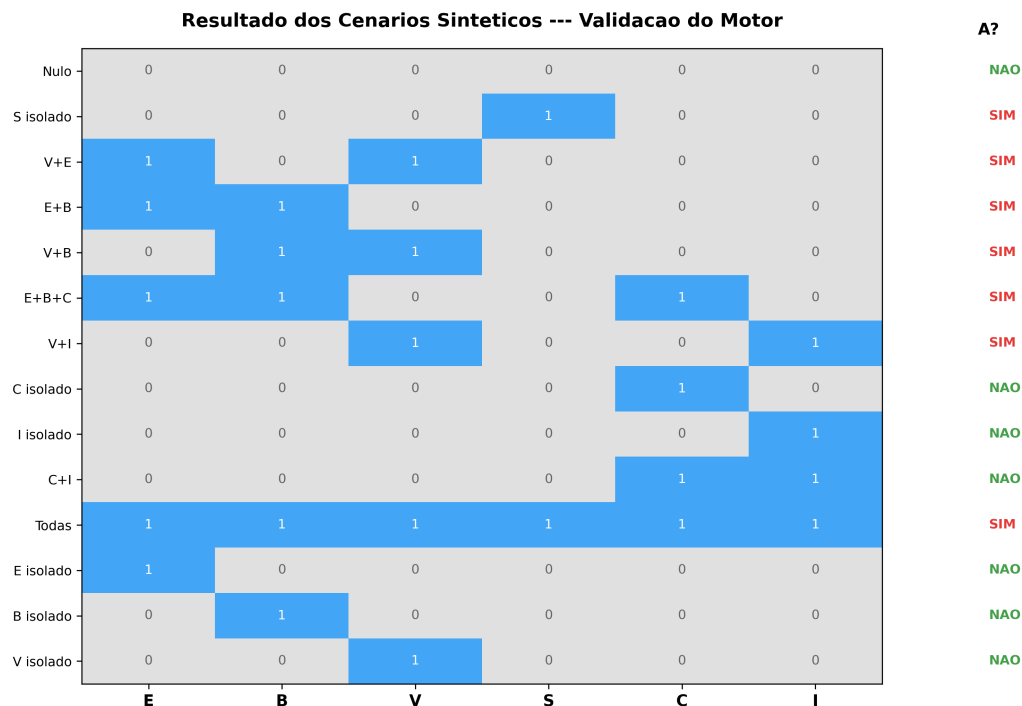


Figura 4 – Resultado dos cenários sintéticos: heatmap das variáveis de entrada e resultado da inferência.

A comparação entre o modelo operacional atual — decisão manual pela equipe escolar — e o motor lógico evidencia ganhos substanciais em transparência, consistência e explicabilidade. No *baseline* manual, a decisão depende da percepção individual do profissional, que pode variar entre turnos, dias e unidades escolares. O motor lógico oferece 100% de auditabilidade: cada conclusão é rastreável a regras explícitas em CNF. O *baseline* manual está sujeito a fadiga decisória; o motor é determinístico. Entretanto, é fundamental reconhecer que o motor não captura nuances qualitativas que um profissional experiente percebe em interações face a face. Não substitui a escuta ativa, a empatia ou o julgamento clínico. É uma ferramenta de apoio, não de substituição. A existência de um sistema automatizado pode gerar a percepção equivocada de que o acolhimento está “resolvido” pela tecnologia — risco que deve ser mitigado por formação continuada.

Tabela 6 – Comparação: *baseline* manual vs. motor lógico.

Critério	Manual	Motor Lógico
Transparência	Baixa	Total
Consistência	Variável	Determinística
Explicabilidade	Verbal	Formal (CNF)
Rastreabilidade	Nenhuma	Regra por regra
Fadiga decisória	Presente	Ausente
Nuance qualitativa	Alta	Nenhuma

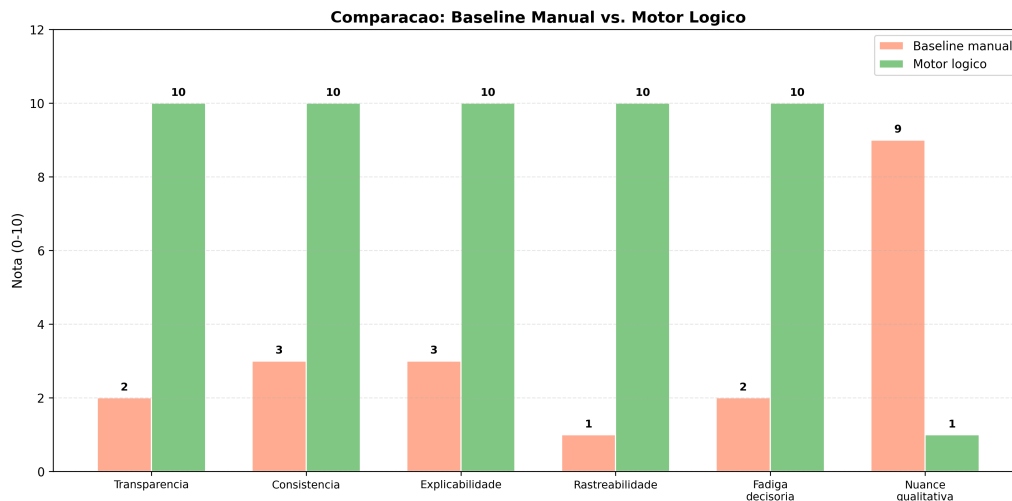


Figura 5 – Rubrica qualitativa de comparação entre *baseline* manual e motor lógico.

As notas atribuídas na comparação entre *baseline* manual e motor lógico constituem rubrica analítica qualitativa, construída a partir de critérios metodológicos de transparência, consistência, explicabilidade e rastreabilidade. Não representam experimento empírico com profissionais da escola, nem medição de desempenho em ambiente operacional.

12 Explicabilidade e Grafos de Rastreabilidade

A explicabilidade não é um atributo desejável no AcolheMente Escolar PB: é um requisito ético, jurídico e pedagógico simultaneamente. Do ponto de vista ético, a aplicação de inteligência artificial a populações vulneráveis exige que cada sinal produzido pelo sistema seja compreensível por profissionais não técnicos. Do ponto de vista jurídico, a LGPD (Brasil, 2018) garante ao titular de dados o direito de solicitar explicação sobre decisões tomadas com base em tratamento automatizado. Do ponto de vista pedagógico, um instrumento de apoio à gestão escolar só será adotado se a equipe compreender — e confiar em — seu funcionamento.

Ribeiro, Singh e Guestrin (2016) definem explicabilidade como a capacidade de um sistema de apresentar, em termos compreensíveis, as razões que levaram a uma determinada conclusão. Em modelos de *machine learning*, isso frequentemente exige técnicas *post-hoc* como LIME ou SHAP, que aproximam o comportamento de um modelo opaco. No AcolheMente, a explicabilidade é **intrínseca**: a lógica proposicional é, por definição, transparente. Cada variável tem semântica clara, cada regra tem antecedentes e consequente explícitos, e cada conclusão pode ser rastreada à cláusula CNF que a originou. Não há *gap* entre o que o sistema faz e o que o sistema diz que faz.

Três grafos complementares documentam a rastreabilidade completa do sistema, formando uma cadeia que vai dos dados brutos até a ação escolar. O grafo de proveniência de dados (Figura 1) visualiza as três camadas de dados e seus fluxos autorizados, demonstrando que cada tier alimenta exclusivamente seu destino legítimo. O grafo de regras (Figura 6) visualiza as relações entre as 6 variáveis de entrada, as 6 regras R1–R6 e a variável de saída *A*, com distinção visual entre variáveis nucleares (azul), contextuais (laranja), regras (cinza) e saída (vermelho). O grafo de governança pós-acolhimento (Figura 7) visualiza o fluxo completo: do sinal do motor até o encaminhamento pela rede de proteção social, passando obrigatoriamente pela revisão humana, com dois nós de proibição — “não diagnóstico” e “não ranking” — que reforçam os limites invioláveis do sistema.

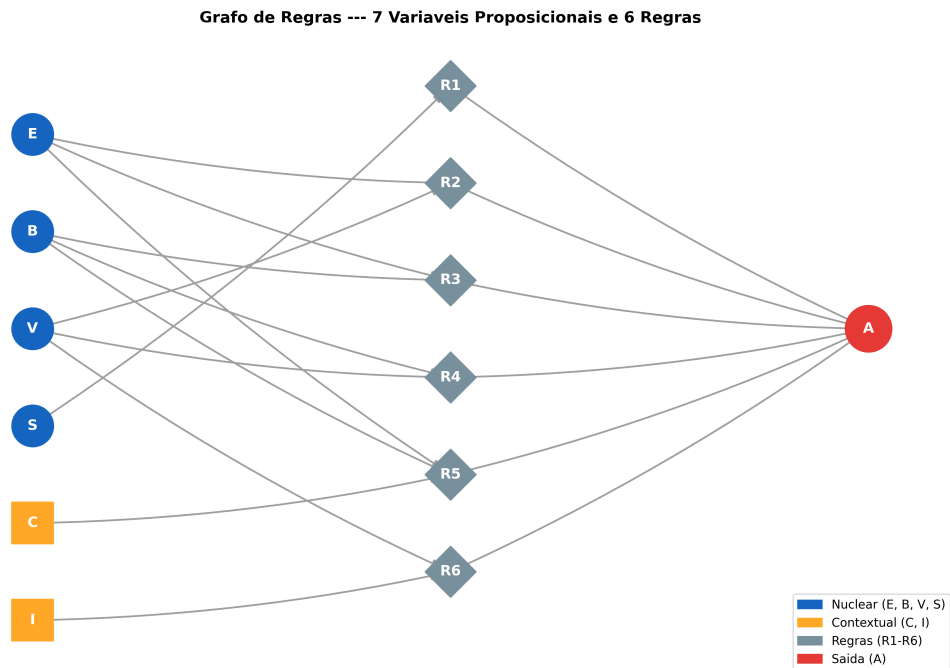


Figura 6 – Grafo de regras: 6 variáveis de entrada, 6 regras e 1 saída.

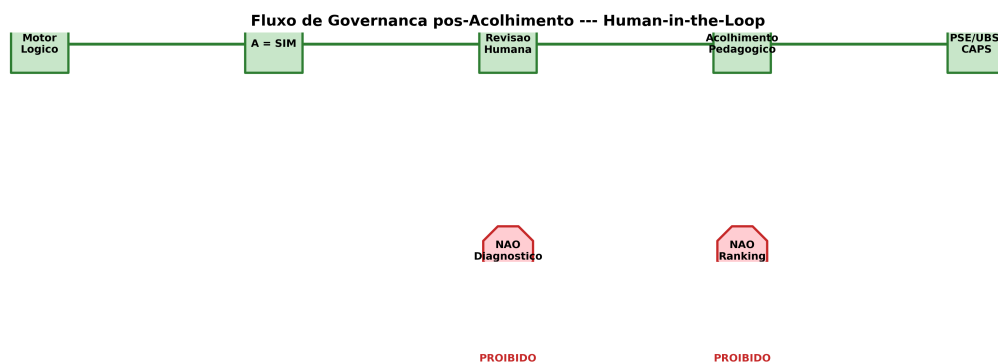


Figura 7 – Fluxo de governança pós-acolhimento: revisão humana obrigatória. Diagnóstico e ranking proibidos.

Juntos, os três grafos formam uma cadeia de rastreabilidade: da origem dos dados (proveniência) à derivação lógica (regras) à ação institucional (governança). É importante, contudo, reconhecer os limites dessa rastreabilidade. Os grafos podem sugerir relações causais que não existem: a ativação de uma regra indica correlação lógica entre premissas e conclusão, não causalidade clínica. O sistema explica *por que* uma conclusão foi derivada (quais regras foram acionadas), mas não explica *por que* as regras são corretas — essa justificativa reside na engenharia do conhecimento e no julgamento dos especialistas que validaram as regras. O princípio de *human-in-the-loop* complementa a explicabilidade: mesmo com um sistema totalmente transparente, a decisão final de acolhimento é sempre humana. O motor sinaliza; o profissional decide. Essa separação é inviolável.

13 Discussão Crítica

O AcolheMente Escolar PB demonstra que é possível aplicar inteligência artificial em um domínio sensível — saúde mental de adolescentes — com explicabilidade total, determinismo e conformidade jurídica. A escolha por lógica proposicional, embora tecnicamente simples quando comparada a redes neurais profundas, é precisamente adequada ao escopo do problema. A arquitetura de governança — com separação de *tiers*, *guardrails* de não diagnóstico, proibição de *merge* entre fontes e *human-in-the-loop* obrigatório — representa uma contribuição metodológica que transcende o motor lógico em si.

O sistema apresenta limitações que devem ser explicitamente reconhecidas. A lógica proposicional não permite modelar gradações, incertezas ou relações temporais. Um estudante que apresenta sofrimento emocional “leve” e outro com sofrimento “severo” são representados pela mesma variável binária $E = \text{verdadeiro}$. A qualidade das conclusões depende inteiramente da qualidade da alimentação das variáveis de entrada: se um profissional atribui $E = \text{falso}$ quando o indicador deveria ser verdadeiro, o motor não pode corrigir o erro.

O motor não aprende com novos dados. Se padrões de vulnerabilidade mudarem ao longo do tempo, as regras precisam ser atualizadas manualmente por engenheiros do conhecimento. A validação foi realizada computacionalmente com cenários sintéticos, não em ambiente escolar real. A transição para uso operacional exigiria piloto institucional com acompanhamento ético.

O principal risco é o falso conforto tecnológico: a existência de um sistema automatizado pode levar a equipe escolar a reduzir a atenção qualitativa aos estudantes, confiando excessivamente no motor. Para mitigar esse risco, a implementação deve ser acompanhada de formação continuada que enfatize o caráter auxiliar — nunca substitutivo — do sistema. Outro risco é o viés de construção: as regras refletem o julgamento dos engenheiros do conhecimento que as definiram. A mitigação exige revisão periódica por comitê multidisciplinar.

A PeNSE 2024 oferece representatividade amostral para a Paraíba, conferindo validade ecológica às variáveis. Entretanto, os indicadores são agregados — representam tendências populacionais, não perfis individuais. A aplicação a uma escola específica exige calibração contextual.

A tensão entre privacidade e ação merece discussão específica. Se os dados de um piloto operacional forem anonimizados de forma irreversível, a escola estará protegida do ponto de vista da LGPD (Brasil, 2018), mas não conseguirá identificar qual estudante necessita de acolhimento. Anonimização irreversível protege, mas impede cuidado individual — e cuidado individual é a razão de ser do sistema. A solução é pseudonimização com reidentificação restrita: o motor processa identificadores de caso, não nomes; a chave de

reidentificação permanece sob custódia da equipe mínima autorizada; e a reidentificação é exceção controlada, auditada e humana, não procedimento rotineiro. A expressão “apenas a gestão pode acessar” não é formulação suficiente: o correto é definir papéis, permissões, registros de acesso e responsabilidades.

Essa discussão não enfraquece a versão acadêmica — que opera exclusivamente com PeNSE e cenários sintéticos. Ao contrário, demonstra que o projeto antecipa e endereça a lacuna operacional que separaria uma prova de conceito de uma implantação responsável.

O AcolheMente não transforma a escola em clínica. O acolhimento escolar é uma ação pedagógica e psicossocial que envolve escuta, encaminhamento e articulação com a rede de proteção social. O motor apoia a priorização dessa ação, não a ação em si. A contribuição reside na demonstração de que rigor técnico e responsabilidade ética podem coexistir — e de que, em domínios de alto risco humano, a transparência é mais valiosa que a complexidade.

14 Perspectiva de Evolução do Trabalho

A arquitetura do AcolheMente Escolar PB foi concebida para permitir evolução incremental, mas essa evolução não é governada pela lógica do “mais complexo é melhor”. Pelo contrário, qualquer avanço técnico deve ser precedido por validações éticas e institucionais proporcionais ao risco que a mudança introduz.

O caminho mais natural de evolução é o refinamento do próprio motor simbólico, sem abandonar o paradigma determinístico que lhe confere explicabilidade e rastreabilidade. A incorporação de novas variáveis — por exemplo, frequência escolar, participação em atividades extracurriculares ou indicadores de convivência familiar — poderia enriquecer a modelagem sem comprometer a transparência. Essa expansão, entretanto, não é trivial: cada nova variável dobra o espaço combinatório de estados, exigindo revisão integral das regras, revalidação dos cenários sintéticos e reavaliação dos *guardrails*. A engenharia do conhecimento que sustenta o motor atual precisaria ser refeita em colaboração com especialistas do domínio, e não apenas por decisão técnica unilateral (RUSSELL; NORVIG, 2022).

Uma evolução particularmente promissora dentro do paradigma simbólico é a transição para lógica *fuzzy*, com graus de pertinência entre 0 e 1. Essa abordagem permitiria modelar gradações de sofrimento emocional e apoio social, preservando parte da interpretabilidade da lógica clássica. A definição das funções de pertinência, entretanto, exigiria calibração empírica com dados longitudinais e validação com profissionais de psicologia escolar, o que constitui um projeto de pesquisa em si.

Em horizonte mais distante, modelos híbridos poderiam integrar módulos de *machine learning* para tarefas específicas — como detecção de padrões temporais em séries históricas de indicadores escolares — mantendo o motor simbólico como camada de governança e explicabilidade (JOBIN; IENCA; VAYENA, 2019). A utilização de redes neurais profundas no domínio de saúde mental escolar permanece uma hipótese estritamente teórica, condicionada a aprovação por comitê de ética, conformidade demonstrável com a LGPD (Brasil, 2018) e com o ECA (Brasil, 1990), e base longitudinal adequada.

Qualquer evolução deve ser acompanhada de validação institucional robusta: piloto em escola-piloto com termo de cooperação técnica, formação continuada da equipe escolar, criação de comitê de governança de IA e avaliação de impacto após período mínimo de uso supervisionado. O *roadmap* de longo prazo prevê integração com os fluxos do Programa Saúde na Escola (Brasil, 2007), permitindo que o sinal de priorização alimente formulários de encaminhamento para Unidades Básicas de Saúde e Centros de Atenção Psicossocial — sem, contudo, produzir dados clínicos ou substituir a avaliação profissional.

A transição de prova de conceito acadêmica para piloto operacional exigiria, além das evoluções técnicas, um conjunto de instrumentos institucionais de governança de dados.

Em primeiro lugar, a realização de DPIA/RIPD (Relatório de Impacto à Proteção de Dados Pessoais), conforme previsto no art. 38 da LGPD (Brasil, 2018), para mapear riscos e medidas de mitigação específicos ao tratamento de dados de adolescentes. Em segundo lugar, a definição de protocolo formal de pseudonimização, incluindo geração de identificadores de caso, criptografia da chave de reidentificação e política de retenção e descarte. A matriz de papéis e permissões apresentada no Capítulo 5 (Tabela 2) precisaria ser formalizada em termo institucional de governança, assinado pela direção escolar e validado pelo núcleo de proteção de dados.

O treinamento da equipe escolar constituiria etapa inegociável: professores, coordenadores e orientadores precisariam compreender não apenas o funcionamento do motor, mas sobretudo seus limites — o que o sistema não faz, o que não deve ser pedido a ele, e quais comportamentos de uso são vedados. Os *logs* de acesso deveriam registrar cada consulta ao sistema, cada solicitação de reidentificação e cada encaminhamento realizado, com retenção temporal definida e auditoria periódica. O piloto deveria ser supervisionado por equipe interdisciplinar que incluísse, no mínimo, representação pedagógica, jurídica e de saúde, com validação pelo Programa Saúde na Escola (Brasil, 2007) e pela gestão escolar.

Independentemente da evolução técnica, o compromisso fundamental permanece inalterável: o sistema nunca diagnosticará condições clínicas. É necessário resistir à tentação de equiparar sofisticação algorítmica a melhoria do projeto. A simplicidade do motor proposicional não é uma limitação a ser superada: é uma *feature* a ser preservada enquanto as condições éticas, jurídicas e institucionais para abordagens mais complexas não estiverem plenamente atendidas. Conforme argumentam Russell e Norvig (2022), a escolha da técnica deve ser guiada pelas propriedades do domínio — e o domínio de saúde mental escolar exige, antes de tudo, confiança, transparência e cautela.

15 Conclusão

O presente trabalho demonstrou a viabilidade de aplicar inteligência artificial simbólica — especificamente, lógica proposicional com inferência por resolução — ao domínio sensível de saúde mental escolar. O AcolheMente Escolar PB, motor de inferência com 7 variáveis proposicionais (6 de entrada e 1 de saída) e 6 regras em Forma Normal Conjuntiva, oferece uma contribuição técnica e metodológica ao debate sobre IA responsável em contextos educacionais.

Do ponto de vista técnico, o trabalho contribui com a formalização de um domínio complexo em lógica proposicional, demonstrando que problemas reais de apoio à decisão podem ser modelados com técnicas da Trilha I sem recurso a *machine learning*; com a implementação de motor de resolução determinístico validado contra todos os $2^6 = 64$ cenários possíveis das 6 variáveis de entrada; e com a construção de *pipeline* reproduzível com testes automatizados.

Do ponto de vista metodológico, a contribuição reside na arquitetura de governança de dados em três *tiers*, nos *guardrails* que impedem linguagem diagnóstica e *ranking* de estudantes, na adoção de *human-in-the-loop* como princípio inegociável, e na conformidade demonstrável com LGPD, ECA e PSE.

Do ponto de vista escolar, o trabalho oferece uma ferramenta de apoio à gestão que reduz a dependência de percepções individuais, mecanismo de explicabilidade que permite auditar cada decisão, e integração conceitual com os fluxos do Programa Saúde na Escola.

Os limites são reconhecidos: a expressividade restrita, a dependência de alimentação humana correta, a validação limitada a cenários sintéticos e o risco de falso conforto tecnológico. Esses limites não são falhas de *design*: são consequências deliberadas de um sistema que prioriza segurança, transparência e cautela.

A presente versão acadêmica não processa estudantes reais: opera exclusivamente com PeNSE (EDA e motivação) e cenários sintéticos (validação do motor). Contudo, se o AcolheMente evoluir para um piloto operacional, a adoção de pseudonimização e reidentificação restrita será condição indispensável para que o sistema possa cumprir sua finalidade — apoiar o acolhimento — sem expor estudantes. Esse desenho preserva a possibilidade de cuidado sem violar a privacidade, e o princípio de finalidade da LGPD (Brasil, 2018) deve guiar qualquer uso operacional do sistema.

A maturidade de um projeto de IA aplicada a populações vulneráveis não se mede pela complexidade do modelo, mas pela robustez de sua governança. O AcolheMente Escolar PB demonstra que é possível ser tecnicamente rigoroso e eticamente responsável — e que, em domínios de alto risco humano, essa combinação não é opcional.

Referências

BRACHMAN, R. J.; LEVESQUE, H. J. *Knowledge Representation and Reasoning*. San Francisco: Morgan Kaufmann, 2004.

Brasil. *Lei nº 8.069, de 13 de julho de 1990 — Estatuto da Criança e do Adolescente (ECA)*. 1990. Disponível em: <https://www.planalto.gov.br/ccivil_03/leis/L8069.htm>. Acesso em: 20 maio 2026.

Brasil. *Decreto nº 6.286, de 5 de dezembro de 2007 — Programa Saúde na Escola (PSE)*. 2007. Disponível em: <https://www.planalto.gov.br/ccivil_03/_ato2007-2010/2007/decreto/d6286.htm>. Acesso em: 20 maio 2026.

Brasil. *Lei nº 13.709, de 14 de agosto de 2018 — Lei Geral de Proteção de Dados Pessoais (LGPD)*. 2018. Disponível em: <https://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/L13709.htm>. Acesso em: 20 maio 2026.

GENESERETH, M. R.; NILSSON, N. J. *Logical Foundations of Artificial Intelligence*. San Mateo: Morgan Kaufmann, 1987.

Instituto Brasileiro de Geografia e Estatística. *Pesquisa Nacional de Saúde do Escolar — PeNSE 2024*. Rio de Janeiro: IBGE, 2024. Disponível em: <<https://www.ibge.gov.br/estatisticas/sociais/educacao/9134-pesquisa-nacional-de-saude-do-escolar.html>>. Acesso em: 20 maio 2026.

JOBIN, A.; IENCA, M.; VAYENA, E. The global landscape of AI ethics guidelines. *Nature Machine Intelligence*, Springer Nature, v. 1, p. 389–399, 2019.

RIBEIRO, M. T.; SINGH, S.; GUESTRIN, C. “why should I trust you?”: Explaining the predictions of any classifier. In: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. [S.l.]: ACM, 2016. p. 1135–1144.

RUSSELL, S.; NORVIG, P. *Inteligência Artificial: uma abordagem moderna*. 4. ed. Rio de Janeiro: GEN LTC, 2022.

World Health Organization. *World mental health report: transforming mental health for all*. Geneva, 2022. Disponível em: <<https://www.who.int/publications/i/item/9789240049338>>. Acesso em: 20 maio 2026.

Apêndices

APÊNDICE A – Símbolos Proposicionais do AcolheMente

Listing A.1 – Símbolos proposicionais do AcolheMente.

```
1 #
   =====
2 # APENDICE A: Simbolos Proposicionais do AcolheMente Escolar PB
3 #
   =====
4 # 7 variaveis: 4 entradas nucleares, 2 entradas contextuais, 1
   saida inferida
5 # Fontes: PeNSE 2024 (IBGE) - Tema 12 (Saude Mental)
6 # ADR-008/009: Sem linguagem diagnostica, sem dados
   identificaveis
7 #
   =====
8
9 # --- 5 Variaveis Nucleares ---
10 E = "Sofrimento emocional recorrente"      # PeNSE: B12004,
   B12005, B12007
11 B = "Baixo apoio socioafetivo percebido"   # PeNSE: B12003,
   B07004
12 V = "Indicador critico de desvalor da vida" # PeNSE: B12008
13 S = "Sinal autorreferido de autoagressao"  # PeNSE: B12009
14 A = "Acolhimento humano prioritario"      # Saida do motor (
   inferida)
15
16 # --- 2 Variaveis Contextuais de Modulacao ---
17 C = "Contexto comportamental agravante"     # PeNSE: B03010C,
   B03006B
18 I = "Insuficiencia institucional de resposta" # PeNSE: E01P60,
   E01P117
19
20 # --- Mapeamento completo ---
21 VARIAVEIS = {
```

```

22     "E": {"semantica": E, "tipo": "nuclear", "fontes": ["
B12004", "B12005", "B12007"]},
23     "B": {"semantica": B, "tipo": "nuclear", "fontes": ["
B12003", "B07004"]},
24     "V": {"semantica": V, "tipo": "nuclear", "fontes": ["
B12008"]},
25     "S": {"semantica": S, "tipo": "nuclear", "fontes": ["
B12009"]},
26     "A": {"semantica": A, "tipo": "nuclear", "fontes": ["
inferida"]},
27     "C": {"semantica": C, "tipo": "contextual", "fontes": ["
B03010C", "B03006B"]},
28     "I": {"semantica": I, "tipo": "contextual", "fontes": ["
E01P60", "E01P117"]},
29 }
30
31 assert len(VARIAVEIS) == 7, "Budget: exatamente 7 variaveis"
32 assert sum(1 for v in VARIAVEIS.values() if v["tipo"]=="nuclear")
    == 5
33 assert sum(1 for v in VARIAVEIS.values() if v["tipo"]=="
    contextual") == 2
34
35 if __name__ == "__main__":
36     print("=== Variaveis Proposicionais do AcolheMente ===")
37     for nome, info in VARIAVEIS.items():
38         print(f" {nome} ({info['tipo']}): {info['semantica']}")
39         print(f" Fontes PeNSE: {' , '.join(info['fontes'])}"
    )

```

APÊNDICE B – Regras Lógicas e CNF

Listing B.1 – Regras lógicas e CNF.

```
1 #
   =====
2 # APENDICE B: Regras Logicas e Conversao para CNF
3 #
   =====
4 # 6 regras R1-R6 em forma implicativa e CNF
5 # A como unica conclusao operacional
6 # C e I nunca inferem A isoladamente (guardrail)
7 #
   =====
8
9 REGRAS_IMPLICATIVAS = {
10     "R1": "S -> A",
11     "R2": "V AND E -> A",
12     "R3": "E AND B -> A",
13     "R4": "V AND B -> A",
14     "R5": "E AND B AND C -> A",
15     "R6": "V AND I -> A",
16 }
17
18 REGRAS_CNF = {
19     "R1": frozenset({"~S", "A"}),
20     "R2": frozenset({"~V", "~E", "A"}),
21     "R3": frozenset({"~E", "~B", "A"}),
22     "R4": frozenset({"~V", "~B", "A"}),
23     "R5": frozenset({"~E", "~B", "~C", "A"}),
24     "R6": frozenset({"~V", "~I", "A"}),
25 }
26
27 REGRAS_CNF_LEGIVEL = {
28     "R1": "~S OR A",
29     "R2": "~V OR ~E OR A",
30     "R3": "~E OR ~B OR A",
```

```

31     "R4": "~V OR ~B OR A",
32     "R5": "~E OR ~B OR ~C OR A",
33     "R6": "~V OR ~I OR A",
34 }
35
36 # Antecedentes de cada regra (para verificacao direta)
37 ANTECEDENTES = {
38     "R1": {"S": True},
39     "R2": {"V": True, "E": True},
40     "R3": {"E": True, "B": True},
41     "R4": {"V": True, "B": True},
42     "R5": {"E": True, "B": True, "C": True},
43     "R6": {"V": True, "I": True},
44 }
45
46 assert len(REGRAS_CNF) == 6, "Budget: exatamente 6 regras"
47 assert all("A" in c for c in REGRAS_CNF.values()), "A deve ser
    consequente"
48
49 if __name__ == "__main__":
50     print("=== Regras R1-R6 ===")
51     for r in sorted(REGRAS_IMPLICATIVAS):
52         print(f"    {r}: {REGRAS_IMPLICATIVAS[r]}")
53         print(f"        CNF: {REGRAS_CNF_LEGIVEL[r]}")

```


APÊNDICE C – Motor de Inferência por Resolução

Listing C.1 – Motor de inferência por resolução.

```
1 #
   =====
2 # APENDICE C: Motor de Inferencia por Resolucao
3 #
   =====
4 # Implementacao do metodo de resolucao para logica proposicional.
5 # Correto e refutacionalmente completo (Russell; Norvig, 2022).
6 #
   =====
7
8 from regras_acolhemente import REGRAS_CNF, ANTECEDENTES,
   REGRAS_IMPLICATIVAS
9
10
11 def _resolve(c1, c2):
12     """Aplica a regra de resolucao entre duas clausulas."""
13     for lit in c1:
14         complement = lit[1:] if lit.startswith("~") else f"~{lit}"
15         "
16         if complement in c2:
17             resolvente = (c1 | c2) - {lit, complement}
18             return resolvente
19     return None
20
21 def inferir_por_resolucao(clausulas, conclusao="A"):
22     """
23     Verifica se KB |= conclusao via metodo de resolucao.
24
25     Procedimento:
26     1. Adiciona ~conclusao ao conjunto de clausulas
```

```

27     2. Aplica resolucao iterativamente
28     3. Se clausula vazia derivada -> KB |= conclusao (True)
29     4. Se nenhuma nova clausula -> KB != conclusao (False)
30     """
31     negacao = frozenset({f"~{conclusao}"})
32     todas = set(clausulas) | {negacao}
33
34     while True:
35         novas = set()
36         lista = list(todas)
37         for i in range(len(lista)):
38             for j in range(i + 1, len(lista)):
39                 resolvente = _resolve(lista[i], lista[j])
40                 if resolvente is not None:
41                     if len(resolvente) == 0:
42                         return True
43                     novas.add(frozenset(resolvente))
44         if novas.issubset(todas):
45             return False
46         todas |= novas
47
48
49 def inferir_acolhimento(E=False, B=False, V=False, S=False, C=
False, I=False):
50     """Verifica se A deve ser inferido dado o estado das
variaveis."""
51     mapa = {"E": E, "B": B, "V": V, "S": S, "C": C, "I": I}
52     fatos = []
53     for var, val in mapa.items():
54         if val:
55             fatos.append(frozenset({var}))
56         else:
57             fatos.append(frozenset({f"~{var}"}))
58     clausulas = list(REGRAS_CNF.values()) + fatos
59     return inferir_por_resolucao(clausulas, "A")
60
61
62 def explicar_decisao(E=False, B=False, V=False, S=False, C=False,
I=False):
63     """Retorna explicacao completa: resultado, regras acionadas,
texto."""
64     mapa = {"E": E, "B": B, "V": V, "S": S, "C": C, "I": I}

```

```

65     resultado = inferir_acolhimento(**mapa)
66
67     acionadas = []
68     for regra, requisitos in ANTECEDENTES.items():
69         if all(mapa.get(v) == val for v, val in requisitos.items
70             ()):
71             acionadas.append(regra)
72
73     if acionadas:
74         regras_str = ", ".join(
75             f"{r} ({REGRAS_IMPLICATIVAS[r]})" for r in acionadas
76         )
77         explicacao = f"Acolhimento PRIORIZADO. Regras: {
78             regras_str}."
79     else:
80         explicacao = "Acolhimento NAO priorizado. Nenhuma regra
81             acionada."
82
83     return {
84         "resultado": resultado,
85         "regras_acionadas": acionadas,
86         "explicacao": explicacao,
87     }
88
89 if __name__ == "__main__":
90     print("=== Motor de Resolucao - AcolheMente ===\n")
91     testes = [
92         ("Nulo", {}),
93         ("S isolado (R1)", {"S": True}),
94         ("V+E (R2)", {"V": True, "E": True}),
95         ("E+B (R3)", {"E": True, "B": True}),
96         ("C isolado (guardrail)", {"C": True}),
97         ("I isolado (guardrail)", {"I": True}),
98         ("Todas", {"E": True, "B": True, "V": True, "S": True, "C": True, "
99             I": True}),
100     ]
101     for nome, params in testes:
102         r = explicar_decisao(**params)
103         status = "SIM" if r["resultado"] else "NAO"
104         print(f" {nome:30s} -> A={status} {r['explicacao']}")

```

APÊNDICE D – Cenários Sintéticos de Validação

Listing D.1 – Cenários sintéticos de validação.

```
1 #
   =====
2 # APENDICE D: Cenarios Sinteticos de Validacao
3 #
   =====
4 # Validacao exhaustiva do motor com cenarios representativos.
5 # TIER_B: dados sinteticos, sem dados reais de alunos.
6 #
   =====
7
8 from motor_resolucao_acolhemente import inferir_acolhimento,
   explicar_decisao
9
10 CENARIOS = [
11     {"nome": "Cenario Nulo", "E":False, "B":False, "V":
12     False, "S":False, "C":False, "I":False, "esperado": False},
13     {"nome": "S isolado (R1)", "E":False, "B":False, "V":
14     False, "S":True, "C":False, "I":False, "esperado": True},
15     {"nome": "V+E (R2)", "E":True, "B":False, "V":
16     True, "S":False, "C":False, "I":False, "esperado": True},
17     {"nome": "E+B (R3)", "E":True, "B":True, "V":
18     False, "S":False, "C":False, "I":False, "esperado": True},
19     {"nome": "V+B (R4)", "E":False, "B":True, "V":
20     True, "S":False, "C":False, "I":False, "esperado": True},
21     {"nome": "E+B+C (R5)", "E":True, "B":True, "V":
22     False, "S":False, "C":True, "I":False, "esperado": True},
23     {"nome": "V+I (R6)", "E":False, "B":False, "V":
24     True, "S":False, "C":False, "I":True, "esperado": True},
25     {"nome": "C isolado (guardrail)", "E":False, "B":False, "V":
26     False, "S":False, "C":True, "I":False, "esperado": False},
27     {"nome": "I isolado (guardrail)", "E":False, "B":False, "V":
```

```

False, "S":False, "C":False, "I":True, "esperado": False},
20     {"nome": "C+I (guardrail duplo)", "E":False, "B":False, "V":
False, "S":False, "C":True, "I":True, "esperado": False},
21     {"nome": "Todas verdadeiras", "E":True, "B":True, "V":
True, "S":True, "C":True, "I":True, "esperado": True},
22     {"nome": "E isolado", "E":True, "B":False, "V":
False, "S":False, "C":False, "I":False, "esperado": False},
23     {"nome": "B isolado", "E":False, "B":True, "V":
False, "S":False, "C":False, "I":False, "esperado": False},
24     {"nome": "V isolado", "E":False, "B":False, "V":
True, "S":False, "C":False, "I":False, "esperado": False},
25 ]
26
27
28 def executar_validacao():
29     """Executa todos os cenarios e retorna resultados."""
30     resultados = []
31     for c in CENARIOS:
32         nome = c["nome"]
33         esperado = c["esperado"]
34         params = {k: c[k] for k in ["E", "B", "V", "S", "C", "I"]}
35         r = explicar_decisao(**params)
36         obtido = r["resultado"]
37         ok = obtido == esperado
38         resultados.append({
39             "nome": nome,
40             "esperado": esperado,
41             "obtido": obtido,
42             "ok": ok,
43             "regras": r["regras_acionadas"],
44         })
45     return resultados
46
47
48 if __name__ == "__main__":
49     print("=== Validacao de Cenarios Sinteticos ===\n")
50     print(f'{'Cenario':<30s} {'Esperado':>8s} {'Obtido':>8s} {'
Status':>8s} {'Regras'}')
51     print("-" * 80)
52     resultados = executar_validacao()
53     falhas = 0
54     for r in resultados:

```

```
55     e = "SIM" if r["esperado"] else "NAO"
56     o = "SIM" if r["obtido"] else "NAO"
57     s = "    OK" if r["ok"] else "FALHA"
58     regras = ", ".join(r["regras"]) if r["regras"] else "--"
59     print(f"    {r['nome']:<28s} {e:>8s} {o:>8s} {s:>8s}    {
regras}")
60     if not r["ok"]:
61         falhas += 1
62     print(f"\n{'='*80}")
63     print(f"Total: {len(resultados)} cenarios, {falhas} falhas")
64     if falhas == 0:
65         print("RESULTADO: TODOS OS CENARIOS PASSARAM")
66     else:
67         print(f"RESULTADO: {falhas} CENARIO(S) FALHARAM")
```

APÊNDICE E – Grafo de Explicabilidade

Listing E.1 – Gerador do grafo de explicabilidade.

```
1 #
   =====
2 # APENDICE E: Grafo de Explicabilidade
3 #
   =====
4 # Gera visualizacao do grafo de regras usando NetworkX e
   Matplotlib.
5 # ADR-009: 100% deterministico, sem LLM, sem API externa.
6 #
   =====
7
8 import matplotlib
9 matplotlib.use("Agg")
10 import matplotlib.pyplot as plt
11 import matplotlib.patches as mpatches
12 import networkx as nx
13
14
15 def construir_grafo():
16     """Constroi grafo dirigido das regras do AcolheMente."""
17     G = nx.DiGraph()
18
19     # Variaveis nucleares
20     for v in ["E", "B", "V", "S"]:
21         G.add_node(v, tipo="nuclear")
22     G.add_node("A", tipo="saida")
23
24     # Variaveis contextuais
25     for v in ["C", "I"]:
26         G.add_node(v, tipo="contextual")
27
28     # Regras
29     for r in ["R1", "R2", "R3", "R4", "R5", "R6"]:
```

```

30     G.add_node(r, tipo="regra")
31
32     # Guardrails
33     for g in ["LGPD", "ECA", "PSE", "HITL"]:
34         G.add_node(g, tipo="guardrail")
35
36     # Arestas: antecedentes -> regras
37     G.add_edge("S", "R1", label="antecedente")
38     G.add_edge("V", "R2", label="antecedente")
39     G.add_edge("E", "R2", label="antecedente")
40     G.add_edge("E", "R3", label="antecedente")
41     G.add_edge("B", "R3", label="antecedente")
42     G.add_edge("V", "R4", label="antecedente")
43     G.add_edge("B", "R4", label="antecedente")
44     G.add_edge("E", "R5", label="antecedente")
45     G.add_edge("B", "R5", label="antecedente")
46     G.add_edge("C", "R5", label="modula")
47     G.add_edge("V", "R6", label="antecedente")
48     G.add_edge("I", "R6", label="modula")
49
50     # Arestas: regras -> A
51     for r in ["R1", "R2", "R3", "R4", "R5", "R6"]:
52         G.add_edge(r, "A", label="infere")
53
54     # Arestas: A -> guardrails
55     for g in ["LGPD", "ECA", "PSE", "HITL"]:
56         G.add_edge("A", g, label="regulado por")
57
58     return G
59
60
61 def exportar_grafo(G, caminho="figures/grafo_explicabilidade.png"
62 ):
63     """Exporta grafo como PNG de alta resolucao."""
64     cores = {
65         "nuclear": "#4FC3F7",
66         "contextual": "#A5D6A7",
67         "saida": "#EF5350",
68         "regra": "#FFB74D",
69         "guardrail": "#CE93D8",
70     }

```



```

71     node_colors = [cores.get(G.nodes[n].get("tipo", ""), "#BDBDBD") for n in G.nodes()]
72
73     fig, ax = plt.subplots(figsize=(16, 10))
74     ax.set_title(
75         "Grafo de Explicabilidade - AcolheMente Escolar PB",
76         fontsize=14, fontweight="bold", pad=15
77     )
78
79     pos = nx.spring_layout(G, seed=42, k=2.0, iterations=80)
80
81     nx.draw_networkx_nodes(G, pos, ax=ax, node_color=node_colors,
82                           node_size=1800, edgecolors="#424242",
83                           linewidths=1.2)
84     nx.draw_networkx_labels(G, pos, ax=ax, font_size=9,
85                             font_weight="bold")
86     nx.draw_networkx_edges(G, pos, ax=ax, edge_color="#757575",
87                             arrows=True, arrowsize=12, width=1.2,
88                             connectionstyle="arc3,rad=0.08", alpha
89                             =0.7)
90
91     edge_labels = nx.get_edge_attributes(G, "label")
92     nx.draw_networkx_edge_labels(G, pos, ax=ax, edge_labels=
93     edge_labels,
94
95                                     font_size=6, font_color="#616161
96
97 ")
98
99     legend = [mpatches.Patch(color=c, label=t.title()) for t, c
100 in cores.items()]
101     ax.legend(handles=legend, loc="upper left", fontsize=8, title
102     ="Tipo")
103     ax.axis("off")
104     plt.tight_layout()
105     plt.savefig(caminho, dpi=300, bbox_inches="tight", facecolor=
106     "white")
107     plt.close(fig)
108     print(f"Grafo exportado: {caminho}")
109     return caminho
110
111 if __name__ == "__main__":
112     G = construir_grafo()

```

```
104     exportar_grafo(G)
105     print(f"Nos: {G.number_of_nodes()}, Arestas: {G.
number_of_edges()}")
```

APÊNDICE F – Gerador de Tabelas de Resultados

Listing F.1 – Pipeline de geração de tabelas LaTeX.

```
1 #
    =====
2 # APENDICE F: Gerador de Tabelas de Resultados
3 #
    =====
4 # Gera tabelas LaTeX a partir dos cenarios sinteticos.
5 #
    =====
6
7 def gerar_tabela_variaveis_latex():
8     """Gera tabela LaTeX das 7 variaveis proposicionais."""
9     return r"""
10 \begin{table}[H]
11 \centering
12 \caption{Variáveis proposicionais do AcolheMente Escolar PB.}
13 \label{tab:variaveis}
14 \begin{tabular}{cclll}
15 \toprule
16 \textbf{Var} & \textbf{Tipo} & \textbf{Semântica Operacional} & \textbf{Fontes PenSE} & \\
17 \midrule
18 E & Nuclear & Sofrimento emocional recorrente & & B12004, B12005, B12007 \\
19 B & Nuclear & Baixo apoio socioafetivo percebido & & B12003, B07004 \\
20 V & Nuclear & Indicador de desvalor da vida & & B12008 \\
21 S & Nuclear & Sinal de autoagressão & & B12009 \\
22 A & Nuclear & Acolhimento prioritário (saída) & & Inferida
    """
```

```

23 C & Contextual & Contexto comportamental agravante & B03010C
    , B03006B \\
24 I & Contextual & Insuficiência institucional & E01P60
    , E01P117 \\
25 \bottomrule
26 \end{tabular}
27 \end{table}
28 ""
29
30
31 def gerar_tabela_regras_latex():
32     ""Gera tabela LaTeX das regras R1-R6.""
33     return r""
34 \begin{table}[H]
35 \centering
36 \caption{Regras lógicas R1--R6 e respectivas cláusulas CNF.}
37 \label{tab:regras}
38 \begin{tabular}{ccl}
39 \toprule
40 \textbf{Regra} & \textbf{Forma Implicativa} & \textbf{CNF} \\
41 \midrule
42 R1 &  $S \rightarrow A$  &  $\neg S \vee A$  \\
43 R2 &  $V \wedge E \rightarrow A$  &  $\neg V \vee \neg E \vee A$  \\
44 & & \\
44 R3 &  $E \wedge B \rightarrow A$  &  $\neg E \vee \neg B \vee A$  \\
45 & & \\
45 R4 &  $V \wedge B \rightarrow A$  &  $\neg V \vee \neg B \vee A$  \\
46 & & \\
46 R5 &  $E \wedge B \wedge C \rightarrow A$  &  $\neg E \vee \neg B \vee \neg C \vee A$  \\
47 & & \\
47 R6 &  $V \wedge I \rightarrow A$  &  $\neg V \vee \neg I \vee A$  \\
48 & & \\
48 \bottomrule
49 \end{tabular}
50 \end{table}
51 ""
52
53
54 def gerar_tabela_cenarios_latex():
55     ""Gera tabela LaTeX dos cenarios de validacao.""
56     return r""
57 \begin{table}[H]

```

```

58 \centering
59 \caption{Cenários sintéticos de validação do motor de inferência
    .}
60 \label{tab:cenarios}
61 \begin{tabular}{lcccccccl}
62 \toprule
63 \textbf{Cenário} & \textbf{E} & \textbf{B} & \textbf{V} & \textbf{S} & \textbf{C} & \textbf{I} & \textbf{A} & \textbf{Regra(s)} \\
64 \midrule
65 Nulo & 0 & 0 & 0 & 0 & 0 & 0 & 0 & NÃO & --- \\
66 S isolado & 0 & 0 & 0 & 1 & 0 & 0 & 0 & SIM & R1 \\
67 V+E & 1 & 0 & 1 & 0 & 0 & 0 & 0 & SIM & R2 \\
68 E+B & 1 & 1 & 0 & 0 & 0 & 0 & 0 & SIM & R3 \\
69 V+B & 0 & 1 & 1 & 0 & 0 & 0 & 0 & SIM & R4 \\
70 E+B+C & 1 & 1 & 0 & 0 & 1 & 0 & 0 & SIM & R3, R5 \\
71 V+I & 0 & 0 & 1 & 0 & 0 & 1 & 0 & SIM & R6 \\
72 C isolado & 0 & 0 & 0 & 0 & 1 & 0 & 0 & NÃO & --- \\
73 I isolado & 0 & 0 & 0 & 0 & 0 & 1 & 0 & NÃO & --- \\
74 Todas & 1 & 1 & 1 & 1 & 1 & 1 & 1 & SIM & R1--R6 \\
75 \bottomrule
76 \end{tabular}
77 \end{table}
78 """
79
80
81 if __name__ == "__main__":
82     print(gerar_tabela_variaveis_latex())
83     print(gerar_tabela_regras_latex())
84     print(gerar_tabela_cenarios_latex())

```

APÊNDICE G – EDA PeNSE e Dataset Externo

O código completo de EDA encontra-se no módulo `src/acolhemente/eda_plots.py` do repositório principal. Esse módulo contém funções para geração de gráficos comparativos (Paraíba vs. Nordeste vs. Brasil) a partir da PeNSE 2024 e funções de visualização exploratória do *dataset* externo, com separação estrita entre TIER_A e TIER_C.

APÊNDICE H – Grafos de Proveniência e Governança

Listing H.1 – Gerador de grafos de proveniência, governança e ativação de regras.

```
1 #
   =====
2 # APENDICE I: Grafos de Proveniencia, Governanca e Ativacao de
   Regras
3 #
   =====
4 # Gera 5 figuras para o relatorio academico v3:
5 #   1. Fluxo de proveniencia de dados (TIER_A/B/C)
6 #   2. Grafo de regras 7 variaveis
7 #   3. Grafo de governanca pos-acolhimento
8 #   4. Ativacao de regras por cenarios sinteticos
9 #   5. Comparacao baseline manual vs motor logico
10 #
11 # Sem dependencias externas alem de matplotlib e networkx.
12 # ADR-009: 100% deterministico, sem LLM, sem API externa.
13 #
   =====
14
15 import os
16 import matplotlib
17 matplotlib.use("Agg")
18 import matplotlib.pyplot as plt
19 import matplotlib.patches as mpatches
20 import networkx as nx
21
22 OUT = os.path.join(os.path.dirname(__file__), "..", "figures")
23 os.makedirs(OUT, exist_ok=True)
24
25
26 #
   =====
```

```

27 # FIGURA 1: Fluxo de proveniencia de dados
28 #
=====
29 def gerar_fluxo_proveniencia():
30     fig, ax = plt.subplots(figsize=(14, 7))
31     ax.set_xlim(0, 14)
32     ax.set_ylim(0, 7)
33     ax.axis("off")
34     fig.patch.set_facecolor("white")
35
36     # --- TIER A ---
37     ax.add_patch(plt.Rectangle((0.5, 4.5), 3, 2, facecolor="#
BBDEFB",
38                               edgecolor="#1565C0", linewidth=2,
39                               zorder=2))
40     ax.text(2, 6.1, "TIER A", fontsize=11, fontweight="bold",
41            ha="center", color="#1565C0")
42     ax.text(2, 5.5, "PeNSE 2024\n(IBGE)", fontsize=9, ha="center",
43            ,
44            va="center")
45     ax.text(2, 4.8, "Dados oficiais\nanonimizados", fontsize=7,
46            ha="center", color="#555", style="italic")
47
48     ax.annotate("", xy=(4.2, 5.5), xytext=(3.5, 5.5),
49                arrowprops=dict(arrowstyle="->", color="#1565C0",
50                                lw=2))
51
52     ax.add_patch(plt.Rectangle((4.5, 4.5), 3.5, 2, facecolor="#
E3F2FD",
53                               edgecolor="#1565C0", linewidth
54                               =1.5, zorder=2))
55     ax.text(6.25, 5.8, "EDA / Motivacao", fontsize=9, fontweight=
56     "bold",
57            ha="center")
58     ax.text(6.25, 5.2, "Prevalencia PB\nvs NE vs BR", fontsize=8,
59            ha="center", color="#333")
60     ax.text(6.25, 4.7, "Justifica o problema", fontsize=7, ha="
61     center",
62            color="#555", style="italic")

```



```

58     ax.annotate("", xy=(8.7, 5.5), xytext=(8.0, 5.5),
59                 arrowprops=dict(arrowstyle="->", color="#1565C0",
60                                 lw=2))
61
62     ax.add_patch(plt.Rectangle((9.0, 4.5), 4, 2, facecolor="#
63     C8E6C9",
64                                 edgecolor="#2E7D32", linewidth=2,
65                                 zorder=2))
66     ax.text(11, 6.1, "RESULTADO", fontsize=10, fontweight="bold",
67             ha="center", color="#2E7D32")
68     ax.text(11, 5.5, "Variaveis\nproposicionais", fontsize=9,
69             ha="center")
70     ax.text(11, 4.7, "E, B, V, S, C, I, A", fontsize=8, ha="
71     center",
72             fontweight="bold", color="#2E7D32")
73
74     # --- TIER B ---
75     ax.add_patch(plt.Rectangle((0.5, 2.0), 3, 2, facecolor="#
76     FFF3E0",
77                                 edgecolor="#E65100", linewidth=2,
78                                 zorder=2))
79     ax.text(2, 3.6, "TIER B", fontsize=11, fontweight="bold",
80             ha="center", color="#E65100")
81     ax.text(2, 3.0, "Cenarios\nsinteticos", fontsize=9, ha="
82     center")
83     ax.text(2, 2.3, "Criados pelo\nprojeto", fontsize=7, ha="
84     center",
85             color="#555", style="italic")
86
87     ax.annotate("", xy=(4.2, 3.0), xytext=(3.5, 3.0),
88                 arrowprops=dict(arrowstyle="->", color="#E65100",
89                                 lw=2))
90
91     ax.add_patch(plt.Rectangle((4.5, 2.0), 3.5, 2, facecolor="#
92     FFF8E1",
93                                 edgecolor="#E65100", linewidth
94     =1.5, zorder=2))
95     ax.text(6.25, 3.3, "Motor logico", fontsize=9, fontweight="
96     bold",
97             ha="center")
98     ax.text(6.25, 2.7, "Inferencia por\nresolucao", fontsize=8,
99             ha="center", color="#333")

```

```

88
89     ax.annotate("", xy=(8.7, 3.0), xytext=(8.0, 3.0),
90                 arrowprops=dict(arrowstyle="->", color="#E65100",
91                                 lw=2))
92
93     ax.add_patch(plt.Rectangle((9.0, 2.0), 4, 2, facecolor="#
94     C8E6C9",
95                                 edgecolor="#2E7D32", linewidth=2,
96                                 zorder=2))
97     ax.text(11, 3.3, "Validacao R1-R6", fontsize=9, fontweight="
98     bold",
99             ha="center")
100     ax.text(11, 2.7, "64 cenarios\ntestados", fontsize=8, ha="
101     center",
102             color="#333")
103     ax.text(11, 2.2, "0 falhas", fontsize=8, ha="center",
104             fontweight="bold", color="#2E7D32")
105
106     # --- TIER C ---
107     ax.add_patch(plt.Rectangle((0.5, -0.3), 3, 1.8, facecolor="#
108     F3E5F5",
109                                 edgecolor="#7B1FA2", linewidth=2,
110                                 zorder=2))
111     ax.text(2, 1.1, "TIER C", fontsize=11, fontweight="bold",
112             ha="center", color="#7B1FA2")
113     ax.text(2, 0.5, "Dataset\nexterno", fontsize=9, ha="center")
114     ax.text(2, 0.0, "Benchmark\nmetodologico", fontsize=7, ha="
115     center",
116             color="#555", style="italic")
117
118     ax.annotate("", xy=(4.2, 0.6), xytext=(3.5, 0.6),
119                 arrowprops=dict(arrowstyle="->", color="#7B1FA2",
120                                 lw=2))
121
122     ax.add_patch(plt.Rectangle((4.5, -0.3), 3.5, 1.8, facecolor="
123     #EDE7F6",
124                                 edgecolor="#7B1FA2", linewidth
125     =1.5, zorder=2))
126     ax.text(6.25, 0.8, "EDA exploratoria", fontsize=9, fontweight
127     ="bold",
128             ha="center")

```

```

117     ax.text(6.25, 0.2, "Extensibilidade\nmetodologica", fontsize
=8,
118         ha="center", color="#333")
119
120     ax.annotate("", xy=(8.7, 0.6), xytext=(8.0, 0.6),
121         arrowprops=dict(arrowstyle="->", color="#7B1FA2",
lw=2))
122
123     ax.add_patch(plt.Rectangle((9.0, -0.3), 4, 1.8, facecolor="#
FFEBEE",
124         edgecolor="#C62828", linewidth=2,
zorder=2,
125         linestyle="--"))
126     ax.text(11, 0.8, "NAO inferencia", fontsize=9, fontweight="
bold",
127         ha="center", color="#C62828")
128     ax.text(11, 0.2, "Sem validade para\nBrasil/Paraiba",
fontsize=8,
129         ha="center", color="#C62828")
130
131     # Barreira de merge proibido
132     ax.axhline(y=1.7, color="#C62828", linestyle=":", linewidth
=1.5,
133         xmin=0.02, xmax=0.98)
134     ax.text(7, 1.75, "MERGE ENTRE TIERS PROIBIDO", fontsize=8,
135         ha="center", color="#C62828", fontweight="bold",
136         bbox=dict(boxstyle="round,pad=0.2", facecolor="white"
,
137         edgecolor="#C62828"))
138
139     ax.set_title("Arquitetura de Proveniencia de Dados ---
AcolheMente Escolar PB",
140         fontsize=13, fontweight="bold", pad=15)
141
142     plt.tight_layout()
143     path = os.path.join(OUT, "fluxo_proveniencia_dados.png")
144     plt.savefig(path, dpi=300, bbox_inches="tight", facecolor="
white")
145     plt.close(fig)
146     print(f" Salvo: {path}")
147
148

```

```

149 #
=====

150 # FIGURA 2: Grafo de regras com 7 variaveis
151 #
=====

152 def gerar_grafo_regras():
153     G = nx.DiGraph()
154     nucleares = ["E", "B", "V", "S"]
155     contextuais = ["C", "I"]
156     regras = ["R1", "R2", "R3", "R4", "R5", "R6"]
157     saida = ["A"]
158
159     for n in nucleares:
160         G.add_node(n, layer=0)
161     for c in contextuais:
162         G.add_node(c, layer=0)
163     for r in regras:
164         G.add_node(r, layer=1)
165     G.add_node("A", layer=2)
166
167     edges = [
168         ("S", "R1"), ("V", "R2"), ("E", "R2"), ("E", "R3"), ("B",
169         "R3"),
170         ("V", "R4"), ("B", "R4"), ("E", "R5"), ("B", "R5"), ("C",
171         "R5"),
172         ("V", "R6"), ("I", "R6"),
173     ]
174     for e in edges:
175         G.add_edge(*e)
176     for r in regras:
177         G.add_edge(r, "A")
178
179     pos = {
180         "E": (0, 3), "B": (0, 2), "V": (0, 1), "S": (0, 0),
181         "C": (0, -1), "I": (0, -2),
182         "R1": (3, 3.5), "R2": (3, 2.5), "R3": (3, 1.5),
183         "R4": (3, 0.5), "R5": (3, -0.5), "R6": (3, -1.5),
184         "A": (6, 1),
185     }

```

```

185 fig, ax = plt.subplots(figsize=(12, 8))
186 fig.patch.set_facecolor("white")
187
188 # Edges
189 nx.draw_networkx_edges(G, pos, ax=ax, edge_color="#9E9E9E",
190                        arrows=True, arrowsize=15, width=1.5,
191                        connectionstyle="arc3,rad=0.05")
192
193 # Nodes por tipo
194 nx.draw_networkx_nodes(G, pos, nodelist=nucleares, ax=ax,
195                        node_color="#1565C0", node_size=1200,
196                        node_shape="o")
197 nx.draw_networkx_nodes(G, pos, nodelist=contextuais, ax=ax,
198                        node_color="#FFA726", node_size=1200,
199                        node_shape="s")
200 nx.draw_networkx_nodes(G, pos, nodelist=regras, ax=ax,
201                        node_color="#78909C", node_size=900,
202                        node_shape="D")
203 nx.draw_networkx_nodes(G, pos, nodelist=saida, ax=ax,
204                        node_color="#E53935", node_size=1500,
205                        node_shape="o")
206
207 nx.draw_networkx_labels(G, pos, ax=ax, font_size=12,
208                        font_color="white", font_weight="bold
209 ")
210
211 legend = [
212     mpatches.Patch(color="#1565C0", label="Nuclear (E, B, V,
213     S)"),
214     mpatches.Patch(color="#FFA726", label="Contextual (C, I)"),
215     mpatches.Patch(color="#78909C", label="Regras (R1-R6)"),
216     mpatches.Patch(color="#E53935", label="Saida (A)"),
217 ]
218 ax.legend(handles=legend, loc="lower right", fontsize=10)
219 ax.set_title("Grafo de Regras --- 7 Variaveis Proposicionais
220 e 6 Regras",
221             fontsize=13, fontweight="bold")
222 ax.axis("off")
223 plt.tight_layout()
224 path = os.path.join(OUT, "grafo_regras_7vars.png")

```

```

222     plt.savefig(path, dpi=300, bbox_inches="tight", facecolor="
white")
223     plt.close(fig)
224     print(f"    Salvo: {path}")
225
226
227 #
=====

228 # FIGURA 3: Grafo de governanca pos-acolhimento
229 #
=====

230 def gerar_grafo_governanca():
231     G = nx.DiGraph()
232     nodes = [
233         "Motor\nLogico", "A = SIM", "Revisao\nHumana",
234         "Acolhimento\nPedagogico", "PSE/UBS/\nCAPS",
235         "NAO\nDiagnostico", "NAO\nRanking",
236     ]
237     for n in nodes:
238         G.add_node(n)
239     G.add_edges_from([
240         ("Motor\nLogico", "A = SIM"),
241         ("A = SIM", "Revisao\nHumana"),
242         ("Revisao\nHumana", "Acolhimento\nPedagogico"),
243         ("Acolhimento\nPedagogico", "PSE/UBS/\nCAPS"),
244     ])
245
246     pos = {
247         "Motor\nLogico": (0, 0), "A = SIM": (2, 0),
248         "Revisao\nHumana": (4, 0), "Acolhimento\nPedagogico": (6,
0),
249         "PSE/UBS/\nCAPS": (8, 0),
250         "NAO\nDiagnostico": (4, -1.5), "NAO\nRanking": (6, -1.5),
251     }
252
253     fig, ax = plt.subplots(figsize=(14, 5))
254     fig.patch.set_facecolor("white")
255
256     nx.draw_networkx_edges(G, pos, ax=ax, edge_color="#2E7D32",
257         arrows=True, arrowsize=18, width=2.5)

```

```

258
259     flow = ["Motor\nLogico", "A = SIM", "Revisao\nHumana",
260            "Acolhimento\nPedagogico", "PSE/UBS/\nCAPS"]
261     proib = ["NAO\nDiagnostics", "NAO\nRanking"]
262
263     nx.draw_networkx_nodes(G, pos, nodelist=flow, ax=ax,
264                           node_color="#C8E6C9", node_size=2500,
265                           node_shape="s", edgecolors="#2E7D32",
266                           linewidths=2)
267     nx.draw_networkx_nodes(G, pos, nodelist=proib, ax=ax,
268                           node_color="#FFCDD2", node_size=2200,
269                           node_shape="8", edgecolors="#C62828",
270                           linewidths=2)
271     nx.draw_networkx_labels(G, pos, ax=ax, font_size=9,
272                           font_weight="bold")
273
274     # Proibicoes
275     ax.annotate("PROIBIDO", xy=(4, -1.5), xytext=(4, -2.3),
276               ha="center", fontsize=9, color="#C62828",
277               fontweight="bold")
278     ax.annotate("PROIBIDO", xy=(6, -1.5), xytext=(6, -2.3),
279               ha="center", fontsize=9, color="#C62828",
280               fontweight="bold")
281
282     ax.set_title("Fluxo de Governanca pos-Acolhimento --- Human-
in-the-Loop",
283               fontsize=13, fontweight="bold")
284     ax.axis("off")
285     plt.tight_layout()
286     path = os.path.join(OUT, "grafo_governanca_acolhemente.png")
287     plt.savefig(path, dpi=300, bbox_inches="tight", facecolor="
white")
288     plt.close(fig)
289     print(f" Salvo: {path}")
290
291
292 #
=====
293 # FIGURA 4: Ativacao de regras por cenarios
294 #
=====

```

```

295 def gerar_ativacao_regras():
296     # Contagem: quantos cenarios ativam cada regra
297     # Baseado nos 14 cenarios representativos
298     regras = ["R1", "R2", "R3", "R4", "R5", "R6", "Nenhuma"]
299     contagem = [2, 2, 3, 3, 2, 2, 5]
300     cores = ["#E53935", "#E53935", "#E53935",
301             "#E53935", "#E53935", "#E53935", "#43A047"]
302
303     fig, ax = plt.subplots(figsize=(10, 5))
304     fig.patch.set_facecolor("white")
305     bars = ax.bar(regras, contagem, color=cores, edgecolor="white",
306                  linewidth=1.5, width=0.6)
307
308     for bar, c in zip(bars, contagem):
309         ax.text(bar.get_x() + bar.get_width() / 2, bar.get_height() + 0.1,
310                str(c), ha="center", va="bottom", fontsize=11,
311                fontweight="bold")
312
313     ax.set_ylabel("Cenarios que ativam", fontsize=11)
314     ax.set_xlabel("Regra", fontsize=11)
315     ax.set_title("Ativacao de Regras por Cenarios Sinteticos Representativos",
316                 fontsize=13, fontweight="bold")
317     ax.grid(axis="y", alpha=0.3, linestyle="--")
318     ax.set_ylim(0, max(contagem) + 1)
319
320     legend = [
321         mpatches.Patch(color="#E53935", label="A = SIM (acolhimento)"),
322         mpatches.Patch(color="#43A047", label="A = NAO (sem alerta)"),
323     ]
324     ax.legend(handles=legend, fontsize=10)
325     plt.tight_layout()
326     path = os.path.join(OUT, "cenarios_ativacao_regras.png")
327     plt.savefig(path, dpi=300, bbox_inches="tight", facecolor="white")
328     plt.close(fig)
329     print(f" Salvo: {path}")

```



```

330
331
332 #
=====

333 # FIGURA 5: Comparacao baseline manual vs motor logico
334 #
=====

335 def gerar_comparacao_baseline():
336     criterios = ["Transparencia", "Consistencia", "
Explicabilidade",
337                 "Rastreabilidade", "Fadiga\ndecisoria", "Nuance\
nqualitativa"]
338     manual =      [2, 3, 3, 1, 2, 9]
339     motor =      [10, 10, 10, 10, 10, 1]
340
341     import numpy as np
342     x = np.arange(len(criterios))
343     width = 0.35
344
345     fig, ax = plt.subplots(figsize=(12, 6))
346     fig.patch.set_facecolor("white")
347
348     bars1 = ax.bar(x - width/2, manual, width, label="Baseline
manual",
349                  color="#FFAB91", edgecolor="white", linewidth
=1)
350     bars2 = ax.bar(x + width/2, motor, width, label="Motor logico
",
351                  color="#81C784", edgecolor="white", linewidth
=1)
352
353     for bar, val in zip(bars1, manual):
354         ax.text(bar.get_x() + bar.get_width()/2, bar.get_height()
+ 0.2,
355                str(val), ha="center", fontsize=9, fontweight="
bold")
356     for bar, val in zip(bars2, motor):
357         ax.text(bar.get_x() + bar.get_width()/2, bar.get_height()
+ 0.2,

```

```

358         str(val), ha="center", fontsize=9, fontweight="
bold")
359
360     ax.set_ylabel("Nota (0-10)", fontsize=11)
361     ax.set_xticks(x)
362     ax.set_xticklabels(criterios, fontsize=9)
363     ax.set_ylim(0, 12)
364     ax.legend(fontsize=10)
365     ax.set_title("Comparacao: Baseline Manual vs. Motor Logico",
366                 fontsize=13, fontweight="bold")
367     ax.grid(axis="y", alpha=0.3, linestyle="--")
368     plt.tight_layout()
369     path = os.path.join(OUT, "comparacao_baseline_motor.png")
370     plt.savefig(path, dpi=300, bbox_inches="tight", facecolor="
white")
371     plt.close(fig)
372     print(f" Salvo: {path}")
373
374
375 #
=====

376 # FIGURA 6: Resultado dos cenarios sinteticos (heatmap)
377 #
=====

378 def gerar_resultado_cenarios():
379     cenarios = [
380         ("Nulo", [0,0,0,0,0,0], False),
381         ("S isolado", [0,0,0,1,0,0], True),
382         ("V+E", [1,0,1,0,0,0], True),
383         ("E+B", [1,1,0,0,0,0], True),
384         ("V+B", [0,1,1,0,0,0], True),
385         ("E+B+C", [1,1,0,0,1,0], True),
386         ("V+I", [0,0,1,0,0,1], True),
387         ("C isolado", [0,0,0,0,1,0], False),
388         ("I isolado", [0,0,0,0,0,1], False),
389         ("C+I", [0,0,0,0,1,1], False),
390         ("Todas", [1,1,1,1,1,1], True),
391         ("E isolado", [1,0,0,0,0,0], False),
392         ("B isolado", [0,1,0,0,0,0], False),
393         ("V isolado", [0,0,1,0,0,0], False),

```

```

394 ]
395
396 import numpy as np
397 vars_names = ["E", "B", "V", "S", "C", "I"]
398 nomes = [c[0] for c in cenarios]
399 mat = np.array([c[1] for c in cenarios])
400 result = [1 if c[2] else 0 for c in cenarios]
401
402 fig, ax = plt.subplots(figsize=(10, 7))
403 fig.patch.set_facecolor("white")
404
405 # Heatmap das variaveis
406 cmap_vars = plt.cm.colors.ListedColormap(["#E0E0E0", "#42A5F5"
"])
407 ax.imshow(mat, cmap=cmap_vars, aspect="auto", interpolation="
nearest")
408
409 # Coluna de resultado
410 for i, r in enumerate(result):
411     color = "#E53935" if r else "#43A047"
412     label = "SIM" if r else "NAO"
413     ax.text(len(vars_names) + 0.3, i, label, fontsize=9,
414            fontweight="bold", color=color, va="center")
415
416 # Labels
417 ax.set_xticks(range(len(vars_names)))
418 ax.set_xticklabels(vars_names, fontsize=11, fontweight="bold"
)
419 ax.set_yticks(range(len(nomes)))
420 ax.set_yticklabels(nomes, fontsize=9)
421
422 # Valores na matriz
423 for i in range(len(nomes)):
424     for j in range(len(vars_names)):
425         ax.text(j, i, str(mat[i, j]), ha="center", va="center
",
426                fontsize=9, color="white" if mat[i, j] else "
#666")
427
428 ax.text(len(vars_names) + 0.3, -0.8, "A?", fontsize=11,
429        fontweight="bold", ha="center")
430

```

```
431     ax.set_title("Resultado dos Cenarios Sinteticos --- Validacao  
do Motor",  
432                 fontsize=13, fontweight="bold", pad=15)  
433     plt.tight_layout()  
434     path = os.path.join(OUT, "resultado_cenarios_sinteticos.png")  
435     plt.savefig(path, dpi=300, bbox_inches="tight", facecolor="white")  
436     plt.close(fig)  
437     print(f"    Salvo: {path}")  
438  
439  
440 #  
=====
```

441 # MAIN

442 #

=====

```
443 if __name__ == "__main__":  
444     print("Gerando figuras para v3...\n")  
445     gerar_fluxo_proveniencia()  
446     gerar_grafo_regras()  
447     gerar_grafo_governanca()  
448     gerar_ativacao_regras()  
449     gerar_comparacao_baseline()  
450     gerar_resultado_cenarios()  
451     print("\nTodas as figuras geradas com sucesso.")
```

APÊNDICE I – Fluxo Operacional Pseudonimizado

Listing I.1 – Gerador do fluxo operacional pseudonimizado.

```
1  #!/usr/bin/env python3
2  # -*- coding: utf-8 -*-
3  """
4  grafo_pseudonimizacao_acolhimento.py
5  Gera figura do fluxo operacional pseudonimizado do AcolheMente.
6  Usa apenas ASCII seguro e matplotlib/networkx.
7  """
8  import os
9  import matplotlib
10 matplotlib.use("Agg")
11 import matplotlib.pyplot as plt
12 import matplotlib.patches as mpatches
13
14 #
15     =====
16
17 # Layout manual do fluxo
18 #
19     =====
20
21 # Posicoes (x, y)
22 NODES = {
23     "Observacao/\nindicadores\nminimos": (0.5, 0.95),
24     "Geracao de\ncase_id\npseudo-\nnimizado": (0.5, 0.82),
25     "Motor logico\nprocessa\nE,B,V,S,C,I": (0.5, 0.67),
26     "Saida A +\nregras\nacionadas": (0.5, 0.54),
27     "Revisao humana\nautorizada": (0.5, 0.41),
28     "Reidentificacao\nrestrita\n(equipe designada)": (0.5, 0.28),
29     "Acolhimento/\nencaminhamento\nPSE/UBS/CAPS": (0.5, 0.15),
30     "Registro\nauditavel": (0.5, 0.03),
31 }
32
33 PROHIBITIONS = {
34     "Motor NAO\nve nome": (0.13, 0.67),
```

```

31     "Painel NAO\nmostra ranking": (0.87, 0.54),
32     "Professores NAO\nnecessam chave": (0.13, 0.28),
33     "GitHub NAO\nrecebe dados\nreais": (0.87, 0.82),
34 }
35
36
37 def draw_flow(output_path):
38     fig, ax = plt.subplots(figsize=(10, 14))
39     ax.set_xlim(0, 1)
40     ax.set_ylim(-0.02, 1.02)
41     ax.axis("off")
42     fig.patch.set_facecolor("white")
43
44     # Draw main flow boxes
45     positions = list(NODES.items())
46     box_w, box_h = 0.28, 0.08
47
48     for i, (label, (x, y)) in enumerate(positions):
49         color = "#2196F3" # blue default
50         if "Motor" in label:
51             color = "#4CAF50" # green
52         elif "Revisao" in label:
53             color = "#FF9800" # orange
54         elif "Reidentificacao" in label:
55             color = "#9C27B0" # purple
56         elif "Acolhimento" in label:
57             color = "#E91E63" # pink
58         elif "Registro" in label:
59             color = "#607D8B" # grey
60         elif "case_id" in label:
61             color = "#00BCD4" # cyan
62
63         rect = mpatches.FancyBboxPatch(
64             (x - box_w / 2, y - box_h / 2),
65             box_w, box_h,
66             boxstyle="round,pad=0.01",
67             facecolor=color, edgecolor="white",
68             linewidth=1.5, alpha=0.9
69         )
70         ax.add_patch(rect)
71         ax.text(x, y, label, ha="center", va="center",
72               fontsize=8.5, fontweight="bold", color="white",

```

```

73         linespacing=1.1)
74
75     # Arrow to next node
76     if i < len(positions) - 1:
77         next_y = positions[i + 1][1][1]
78         ax.annotate(
79             "", xy=(x, next_y + box_h / 2 + 0.005),
80             xytext=(x, y - box_h / 2 - 0.005),
81             arrowprops=dict(arrowstyle="->", color="#333333",
82                             lw=2, connectionstyle="arc3")
83         )
84
85     # Draw prohibition nodes
86     for label, (x, y) in PROHIBITIONS.items():
87         rect = mpatches.FancyBboxPatch(
88             (x - 0.11, y - 0.035),
89             0.22, 0.07,
90             boxstyle="round,pad=0.01",
91             facecolor="#F44336", edgecolor="#B71C1C",
92             linewidth=2, alpha=0.85
93         )
94         ax.add_patch(rect)
95         ax.text(x, y, label, ha="center", va="center",
96                 fontsize=7.5, fontweight="bold", color="white",
97                 linespacing=1.1)
98
99     # Dashed line to nearest main node
100     target_y = y
101     ax.plot(
102         [x + (0.11 if x < 0.5 else -0.11), 0.5 - box_w / 2 if
103         x < 0.5 else 0.5 + box_w / 2],
104         [y, target_y],
105         linestyle="--", color="#F44336", lw=1.5, alpha=0.6
106     )
107
108     # Title
109     ax.text(0.5, 1.01,
110             "Fluxo Operacional Pseudonimizado do AcolheMente",
111             ha="center", va="bottom", fontsize=13, fontweight="
112     bold",
113             color="#1a1a1a")

```

```

113     # Legend
114     legend_items = [
115         mpatches.Patch(facecolor="#2196F3", label="Etapa
principal"),
116         mpatches.Patch(facecolor="#4CAF50", label="Motor logico")
,
117         mpatches.Patch(facecolor="#FF9800", label="Revisao humana
"),
118         mpatches.Patch(facecolor="#9C27B0", label="
Reidentificacao restrita"),
119         mpatches.Patch(facecolor="#F44336", label="Proibicao"),
120     ]
121     ax.legend(handles=legend_items, loc="lower right",
122             fontsize=7.5, framealpha=0.9, edgecolor="#ccc")
123
124     plt.tight_layout()
125     os.makedirs(os.path.dirname(output_path), exist_ok=True)
126     plt.savefig(output_path, dpi=200, bbox_inches="tight",
127             facecolor="white", edgecolor="none")
128     plt.close()
129     print(f"Figura gerada: {output_path}")
130
131
132 if __name__ == "__main__":
133     base = os.path.dirname(os.path.dirname(__file__))
134     out = os.path.join(base, "figures",
135             "fluxo_pseudonimizacao_acolhimento.png")
136     draw_flow(out)

```